جامعة أبوظبي
Abu Dhabi University

College of Engineering

Spring 2025-2026

Course Name and code	**MEC483 – Mechatronics System Design**		
Instructor	Dr. Claudio Vignola		
Assignment/ Project Title and number	**Group Project – Autonomous Pick & Place Robot**		
Due date	15 June 2026		
Submission Date	15 June 2026		
Submitted by: **Student Name** **Student ID** **Signature**	1. Maher Abo Abed	1087993	
	2. Sabeeha Zainab Hasham	1089042	
	3. Basel Feras	1088912	
	4. Ahmed Nasser Alshehhi	1090822	

Table of Contents

1. Business Model Canvas 1
2. Problem Definition 2
   2.1 Final Problem Statement 2
   2.2 Final Impact Statement 2
   2.3 Criteria 3
   2.4 Constraints 4
3. Project Management Overview 5
   3.1 Team Members Responsibilities & Accomplishments 5
   3.2 Gantt Chart 7
4. Hardware Design Narrative 7
   4.1 Final Electrical Block Diagram 8
   4.2 Detailed Hardware Description 9
   4.3 Final Major Component Selection Rationale 11
      4.3.1 Microcontroller Selection 11
      4.3.2 Camera Selection 12
      4.3.3 Mini-Computer (Vision Process) Selection 13
      4.3.4 Arm Actuator 14
   4.4 MG995 Torque Sizing for the Arm 16
   4.5 Power Supply Selection 16
   4.6 Power Budget 17
   4.7 Final Electrical Schematic 18
5. Software Design Narrative 19
   5.1 Software Design Rationale 19
   5.2 Vision Detection System 19
      5.2.1 Dataset Acquisition 20
      5.2.2 Image Annotation and Labelling 21
      5.2.3 Training the YOLOv8 Model 23
      5.2.4 HSV Based Object Detection 25
      5.2.5 Integration and Deployment 26
      5.2.6 Shape Detection 29
      5.2.7 Distance Vision 32
   5.3 Web-Based Robot Controller Interface 35
6. Mechanical Design Narrative 39
   6.1 Arm Structure 39

6.2 Base Platform 41
6.3 Dimensions and Workspace Envelope 42
7. Inverse Kinematic Narrative 43
7.1 Robot Modelling and MoveIt Setup 43
7.2 Resolving Initial IK Failures 47
7.3 Position-Only IK and Workspace Validation 47
8. Version 2.0 and Lesson Learned 49
9. References 51
10. Appendix 52

Table of Figures

Figure 1-1: Business Model Canvas for the Pick and Place Robot 1
Figure 3-1: Initial Gantt chart. 7
Figure 4-1: The final electrical block diagram of the system. 8
Figure 4-2: All the hardware parts used in the making of the robotic arm. 9
Figure 4-3: Simulations with a payload of 250g confirm the choice of the motors. 16
Figure 4-4: Electrical schematic of the system. Red shows 12-6 V power lines, black is for ground, blue is for servos signal lines, and orange is for 5 V VCC line. 19
Figure 5-1: Egg (left) and Nut (right) Datasets used for Detection 20
Figure 5-2: Bounding Box Annotations used for Training 22
Figure 5-3: Txt File for Egg Detection (1) 23
Figure 5-4: YOLOv8 Training using PyCharm in a Custom Dataset 24
Figure 5-5: HSV Calibration Tool for Green Object Detection 25
Figure 5-6: Successful Detection During Live Camera Testing of the Trained Model 27
Figure 5-7: YOLOv8 Model Deployed on the Raspberry Pi 28
Figure 5-8: Green ball detection with distance estimation. 34
Figure 5-9: Compass detection with distance estimation. 34
Figure 5-10: Entry page of the Pick and Place Robot Controller. 35
Figure 5-11: Robot Operation Center showing connection state, operation modes, and safety controls. 36
Figure 5-12: Manual Arm and Manual Platform control panels. 37
Figure 5-13: Camera stream, command log, and robot status panel. 38
Figure 6-1: Circular servo horn used for the joint fitting. 39

Figure 6-2: 3D CAD render of the robotic arm in SolidWorks. 40
Figure 6-3: 3D CAD render of the gripper mechanism. 40
Figure 6-4: Physical assembly of robotic arm. 41
Figure 6-5: Fully assembly robotic arm attached to the platform. 42
Figure 7-1: SolidWorks used to define joint axis and create the URDF model based on the CAD assembly. 44
Figure 7-2: Defining the kinematic chain. 44
Figure 7-3: Defining joint limits for the URDF model. 45
Figure 7-4: MoveIt2 Setup Assistant. 45
Figure 7-5: Configuring the MoveIt2 solver. 46
Figure 7-6: Independent control of individual joints using sliders to determine angular position. 46
Figure 7-7: Workspace scan to validate inverse kinematic solver. 48

Table of Tables

Table 2-1: Pick and Place Robot Project Criteria 3
Table 2-2: Constrains within the Project 4
Table 4-1: Summary of all the components used and their function in the system. 9
Table 4-2: Weighted decision matrix - microcontroller selection. 11
Table 4-3: Weighted decision matrix - camera selection. 12
Table 4-4: Weighted decision matrix - minicomputer selection. 14
Table 4-5: Decision matrix - robotic arm actuator. 15
Table 4-6: Power budget summary. 17
Table 6-1: Robot footprint and arm reach envelope. 43
Table 7-1: Sample of confirmed reachable workspace points. 48

1. Business Model Canvas

The business model canvas was developed in order to evaluate the commercial feasibility and potential implementation strategy of the proposed Pick and Place Robotic System. The canvas identifies the key stakeholders, resources, activities and value propositions required to successfully develop and maintain the system. A structured overview of how the project can create value for customers whilst maintaining financial performance was provided. The model considers both the technical capabilities of the system alongside the practical requirements for its deployment within manufacturing, logistics and potential educational environments.

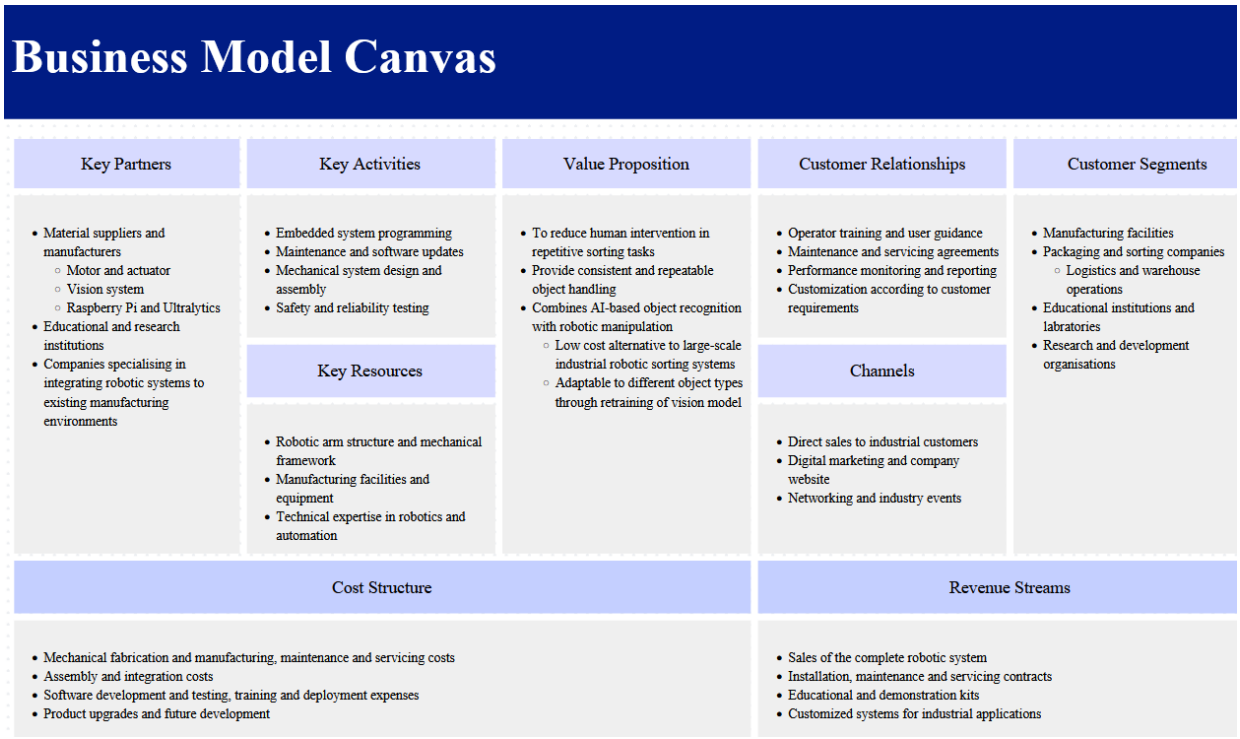

Figure 1-1: Business Model Canvas for the Pick and Place Robot

The analysis demonstrates that the proposed system offers value through automation, improved operational consistency, and the reduced dependence on manual labour. The identified customer segments can indicate the potential applications across a series of disciplines such as manufacturing, packaging, logistics and educational sectors. Strategic partnerships with component suppliers, automation integrators and educational institutions support both the system development and future scalability. The revenue streams and cost structure suggest a more feasible pathway for commercialization through system sales, integration services and support contracts that will be ongoing. The design provides a foundation for future enhancements such as higher speed operations and additional object classes; this system supports growing technological fields through intelligent automation and data driven decision

making. Overall, the business model canvas highlights the potential of the pick and place robotic system to evolve simply from a prototype into a more practical automation solution for a range of industrial and educational applications.

2. Problem Definition

2.1 Final Problem Statement

The modern manufacturing and automation systems demand fast, accurate and reliable object handling process. Manual pick and place operations can be slow, and human errors are more susceptible to occur, they are also not suitable for repetitive high precision operations. In addition, traditional robotic systems are typically designed to work with objects in a fixed location, and they cannot adapt to changing environments.

To tackle these challenges, this project proposes to build a Smart Pick and Place Robotic System with computer vision integrated within it. The system is aimed at automatically identifying objects in a workspace, locating their positions and operating a robot arm to pick and place the objects without any human intervention. The system combines sensing, image processing, embedded control and mechanical actuation offering a smart and efficient automation solution.

2.2 Final Impact Statement

The developed system shows the application of mechatronic engineering principles with the integration of mechanical, electrical, control and computer engineering subsystems. The combination of computer vision in the robotic arm enhances the precision of object detection and reduces reliance on manual handling, improving both accuracy and efficiency.

The project helps to understand the latest automation technologies utilized in manufacturing, warehousing, packaging and assembly processes. Moreover, the platform developed will serve as a basis for future developments like advanced object recognition and fully autonomous industrial robotic systems.

2.3 Criteria

Table 2-1: Pick and Place Robot Project Criteria

Criteria	Rationale	Electrical Implementation	Software Implementation	Mechanical Implementation
CR-1: Object Detection Accuracy	Ensure reliable object identification	Stable camera power and communication interfaces	Computer vision algorithm detects and identifies objects	Proper camera positioning and workspace setup
CR-2: Pick-and-Place Success Rate	Evaluate overall system effectiveness	Reliable motor drivers and actuator control circuits	Accurate object coordinate processing	Stable robotic arm structure and gripper design
CR-3: Positioning Accuracy	Ensure precise object placement	Stable power delivery	Calibration of each component	Accurate joint assembly and minimized mechanical tolerances
CR-4: Response Time	Improve operational efficiency	Fast signal transmission between components	Efficient image processing and serial communication	Smooth robotic arm motion with minimal delay
CR-5: Payload Capacity	Ensure practical object handling capability	Motors supplied with adequate voltage and current	Motion control adapted to payload requirements	Strong arm structure and gripper mechanism
CR-6: Repeatability	Consistent performance over multiple cycles	Stable electrical operation and sensor readings	Repeatable control commands and calibration procedures	Rigid joints with minimal backlash
CR-7: Workspace Coverage	Ensure sufficient reach within workspace	Proper actuator selection and power distribution	Workspace mapping and coordinate transformation	Arm dimensions optimized for maximum reach

2.4 Constraints

Table 2-2: Constrains within the Project

Constraints	Rationale	Electrical Implementation	Software Implementation	Mechanical Implementation
CO-1: Prototype Budget	Limited student project budget	Cost-effective electronic components selected	Open-source software tools used	Low-cost manufacturing materials used
CO-2: Power Supply	Provide stable operation of all subsystems	6V for motors and 5V for electronics using buck converters	Stable operation of vision and control software	Supports actuator and sensor requirements
CO-3: Microcontroller	Required embedded control platform	Arduino controls motors and sensors	Communication with vision system	Interfaces with robotic arm actuators
CO-4: Computer Vision Integration	Enable autonomous object recognition	Servo motors connected to Arduino outputs	Motion control routines implemented	Robotic arm joints actuated for movement
CO-5: Actuators Controlled by Microcontroller	Enable robotic arm motion	Servo motors connected to Arduino outputs	Motion control routines implemented	Robotic arm joints actuated for movement
CO-6: Serial Communication	Transfer data between vision system and controller	USB communication interface	Python-Arduino communication implemented	Supports coordinated arm movement
CO-7: Manufacturing Method	Limited fabrication resources	Proper mounting of electronics	Software compatible with hardware layout	Laser-cut platform and fabricated arm components
CO-8: Safe Operation	Ensure safe laboratory operation	5V and 6V regulated power supplies	Software limits prevent unsafe movements	Stable structure and safe moving parts
CO-9: Project Timeline	Course requirement	Electronics integrated on schedule	Software tested and debugged on schedule	Mechanical assembly completed on schedule

3. Project Management Overview

3.1 Team Members Responsibilities & Accomplishments

Maher Abo Abed:

Maher Abo Abed was the project's group leader, responsible for monitoring the progress of the project and the embedded systems, electrical and mechanical integration work. Electrically, Maher did the wiring for the system, which involved connecting the common ground between the microcontroller, the servo driver, the motor driver and the external power supply, after determining that the electrical supply sagged when the servos were loaded, necessitating the use of a separate power supply for the servo and motor rails. Maher adjusted each arm joint individually for motor configuration, from angle-based to direct pulse-width control for finer adjustment. He set the working pulse ranges for the elbow and wrist (including correcting for the non-standard dead zone of the wrist) and solved servo jitter on unused driver channels while creating a serial command interface for fast, iterative calibration. Maher programmed the firmware for joint control, encoder reading, motor driving and a GUI for joint control via serial. He also worked on the inverse kinematics development using ROS 2 and MoveIt 2, creating the kinematic model of the arm, configuring the MoveIt planning pipeline, fixing the IK solver problems that initially prevented the IK, and testing the arm's workspace with a custom scanning node. In manufacturing and assembly, Maher designed and 3D printed the arm's links and joints using PLA, as well as the servo mounts and gripping/suction mechanism. In the process, he discovered that the originally designed shoulder joint was redundant with the steering capability of the mobile base, and reduced the arm to a 3-DOF manipulator. He also cut the chassis from acrylic with a laser and mounted the motors, arm and electronics on it. During the project, Maher performed a significant amount of testing at the subsystem and system level, individual servo and joint calibration, motor and encoder testing, workspace reachability testing, and integration testing of the wireless control link, and coordinated the team's documentation (including the hardware design, component selection, power budget, and inverse kinematics sections of the final report).

Sabeeha Hasham:

Sabeeha Hasham’s main role within this project was the vision detection system, this involved obtaining the dataset alongside training and annotating the images that were taken. She focused on making HSV work for one of the objects and perfecting the dataset for the remaining four objects, this was carried out by selecting an appropriate camera and connecting it alongside uploading the trained YOLOv8 model to the Raspberry Pi therefore enabling the set to be tested

and issues like lagging and sensitivities to be altered based on the requirements of the project. The images taken were manually annotated and categorized into her laptop to then be trained over 100 cycles ensuring that the model understood the bounding regions of the object in a series of different environments. Issues were faced with the camera selection as with the Pi Camera, viewing the output on the pi was not successful therefore it had to be changed, continuous issues with the model were faced as well. Throughout the end of the process where integrating the components together had to occur, she had some involvement in the testing alongside mitigating further issues faced. The various resources and software used throughout this project were: Anaconda, PyCharm, Ubuntu, CVAT, PowerShell, OpenCV in order to train the set and obtain images from open source resources alongside transferring resources such as the best.pt file required for identification of the objects within a space.

Basel Feras Ghunaim:

Basel Feras Ghunaim designed and controlled the suction system, designed the gripper, designed the distance estimation pipeline using the camera, designed the wireless communication link, designed the project website, and did some aspects of system integration and testing. Basel developed the pneumatic circuit and chose the air pump for the suction system and began to work on the control of the air pump. After testing the pump with a MOSFET, he decided to use a relay as the switching device as it was more reliable and easier to control. He also created the rack and pinion gripper mechanism that is used in conjunction with the suction cup in the hybrid end effector. On the computer vision side, Basel developed an object-tracking model that allows them to calculate the distance of an object detected by the camera from the camera itself. This included increasing the training set to make it more robust for detection, and using reference images to calibrate the relationship between the apparent size of the object in the image and the distance of the object in real life, then using mathematical formulas based on this calibration to translate the size measured in the image into the distance to the object. For the streaming pipeline, Basel fine-tuned the frame rate, quality and resolution of the camera stream to optimise the accuracy of the detection versus the processing time, and set up the Raspberry Pi to act as a Wi-Fi bridge between the camera, the detection system and the rest of the network. Basel handled the wiring and configuration of the HC-05 module to create the Bluetooth communication link between the Arduino and the control system, collaborating with Maher to configure the module and connect the robot to the operator's computer to allow the exchange of commands and status updates between the two wirelessly. Basel also designed, developed, and integrated the project's control website, while Maher was

involved in its implementation. Basel has played a key role in testing and experimentation at the component and system level throughout the project, provided overall system integration support, and made a contribution to the project documentation.

Ahmed Alshehhi:

Ahmed Alshehhi was involved in the design of the control system application and the integration of the project subsystems in order to make sure that the hardware and software components are communicating seamlessly. He helped to wire the electronic components, check the systems connections and help with the overall integration process. Additionally, Ahmed was involved in project file management and collaboration using GitHub, ensuring that project files and documentation remain organized and version controlled.

He also helped make 3D printed components and assembly of the robotic arm structure. Engaged in system trials, testing, and troubleshooting to assess the performance of prototypes and make revisions. Ahmed also helped in the documentation of the project throughout and the preparation of the final report.

3.2 Gantt Chart

PROJECT: Smart Pick-and-Place Robot

Legend: On Track | Low Risk | Med Risk | High Risk

Project Start: Feb 24, 2026 | Project End: Jun 08, 2026

Task	Category	Assigned To	Progress	Start	Days	Feb 24 W1	Mar 03 W2	Mar 10 W3	Mar 17 W4	Mar 24 W5	Mar 31 W6	Apr 07 W7	Apr 14 W8	Apr 21 W9	Apr 28 W10	May 05 W11	May 12 W12	May 19 W13	May 26 W14	Jun 02 W15
System Design and Hardware																				
System Definition and Primary Sensing	On Track	Group		Feb 24	5															
Embedded Controller Selection	On Track	Maher		Mar 1	4															
Electrical Actuation and Motor Selection	On Track	Sabeeha		Mar 5	10															
CAD Design	On Track	Maher & Sabeeha		Mar 24	7															
Component Procurement	On Track	Maher, Basel, Ahmed		Mar 24	14															
Electrical Schematic & Wiring Plan	On Track	Group		Mar 24	7															
Fluid Power & Kinematic Chain	On Track	Group		Mar 10	10															
Signal Processing and Data Handling																				
Sensor Integration & Calibration	On Track	Basel & Sabeeha		Mar 15	25															
Signal Conditioning & Conversion	On Track	Basel & Ahmed		Mar 15	12															
Digital Logic & Data Presentation	On Track	Ahmed		Mar 20	10															
I/O Interfacing & Industrial Control	On Track	Maher & Sabeeha		Mar 25	12															
Control Systems and Implementation																				
Mechanical Assembly & Electrical Wiring	On Track	Group		Mar 31	7															
Firmware Development	On Track	Group		Apr 1	18															
Mathematical System Modelling	On Track	Maher		Apr 5	12															
Stability & Feedback Control	On Track	Basel		Apr 15	10															
Communication and Final Integration																				
Communication & Networking	On Track	Ahmed		Apr 25	10															
Intelligent Control & Final Design	On Track	Group		May 5	20															
Testing and Final Phase																				
System Integration Testing	On Track	Group		May 25	7															
Debugging & Optimization	On Track	Group		Jun 1	5															
Final Demo & Documentation	On Track	Group		Jun 6	3															

Figure 3-1: Initial Gantt chart.

4. Hardware Design Narrative

This section covers the hardware side of the project: how the wiring is laid out, what each part does, why we picked the parts we did, and how power is supplied and budgeted across the system. At a high level, the robot is made up of four pieces working together: a wheeled base that moves the robot around,

a 4-jointed arm that picks things up, a camera-based vision system that finds objects, and a wireless link that lets us send commands and see what the robot sees.

4.1 Final Electrical Block Diagram

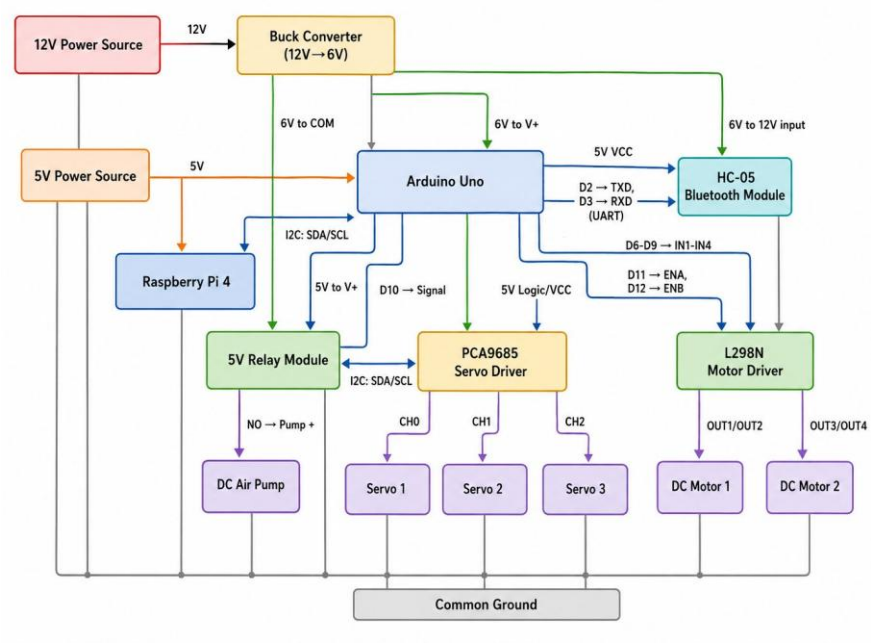

Figure 4-1: The final electrical block diagram of the system.

The above figure gives an overview of how the different components are connected to each other. A 12V battery supply is sent through a step-down buck converter to reduce the voltage to 6V. The output of the buck converter is used to power the relay through the COM port, the PCA9685 driver, pump, and the l298N motor driver which operates two dc motors. Another line is used which has a power rating of 5V, this power line is used as a source for the Arduino and Raspberry Pi. This voltage comes from a regulated source. 5V is also used for all the signal lines (VCC/IN), this is seen in the relay, HC05, and PC9685. All the components are grounded to one common ground as to keep the signal clean and reduce any electrical noise.

4.2 Detailed Hardware Description

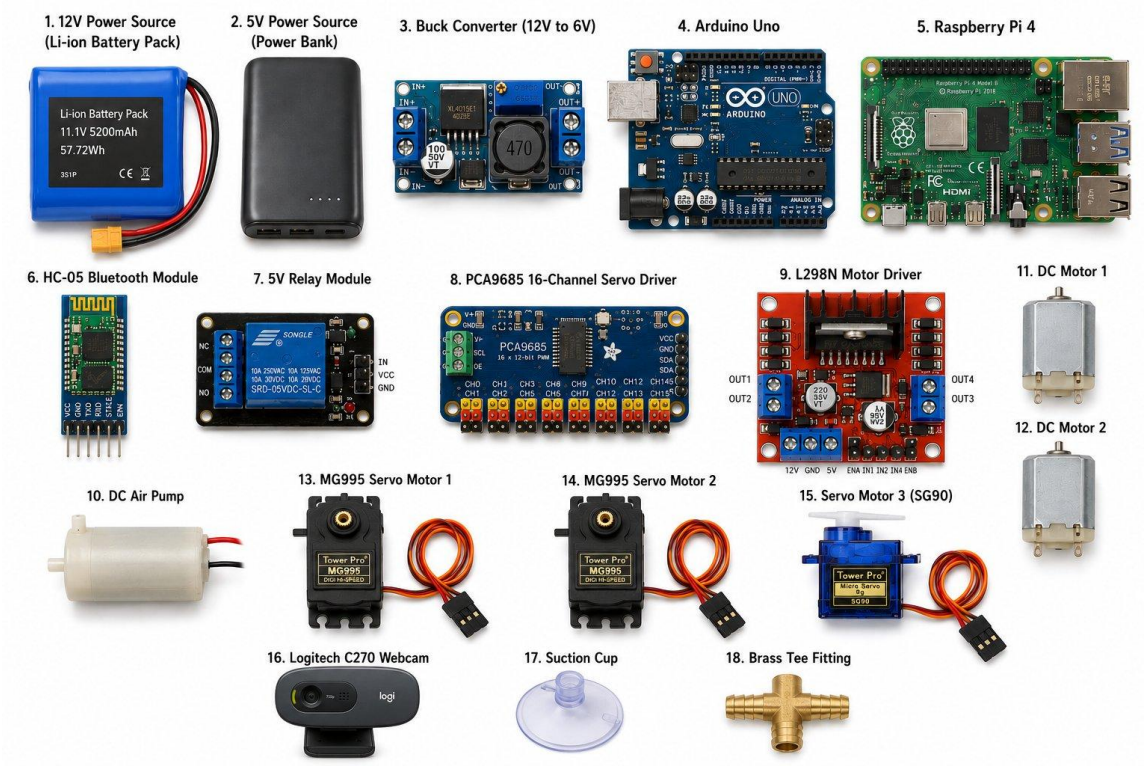

Figure 4-2: All the hardware parts used in the making of the robotic arm.

Table 4-1: Summary of all the components used and their function in the system.

No.	Component	What It Does in the System
1	12 V Power Source (Li-ion Battery Pack)	The main battery. It powers the buck converter, which then supplies the actuators, relay and pump.
2	5 V Power Source (Power Bank)	Gives Raspberry Pi and Arduino its own steady 5 V supply, so it does not lose power when the servos or motors draw a lot of current.
3	Buck Converter (12 V → 6 V)	Steps the 12 V battery voltage down to a steady 6 V for the actuators, relay and pump.
4	Arduino Uno	The main controller. It reads the signals and data input, sends commands to the servos through the driver board, and drives the two-wheel motors.
5	Raspberry Pi 4	Used as bridge between the camera and the laptop, to run YoloV8 (computer vision model).

6	HC-05 Bluetooth Module	A wireless link between the Arduino and the computer running the dashboard, used to send commands and receive status updates.
7	5 V Relay Module	Acts as a switch that the Arduino uses to turn the air pump on and off.
8	PCA9685 16-Channel Servo Driver	Generates the signals needed to control the three arm servos (wrist, elbow, gripper), freeing up the Arduino's limited number of pins.
9	L298N Motor Driver	Provides enough current to drive the two wheel motors, since the Arduino's pins cannot supply that much power on their own.
10	DC Air Pump	Creates suction for the gripper's suction cup, allowing it to pick up objects.
11	DC Motor 1 (with encoder)	Drives the left wheel of the mobile base. The encoder lets the Arduino know how fast and how far the wheel has turned.
12	DC Motor 2 (with encoder)	Drives the right wheel of the mobile base, with the same encoder feedback as Motor 1.
13	MG995 Servo Motor 1	A strong servo used for the wrist joint of the arm.
14	MG995 Servo Motor 2	A strong servo used for the elbow joint of the arm.
15	Servo Motor 3 (SG90)	A smaller, lighter servo used to open and close the gripper.
16	Logitech C270 Webcam	The camera used for object detection , both the YOLOv8 model and the colour-based detection rely on this camera's video feed.
17	Suction Cup	The part of the gripper that actually touches and holds onto objects, using suction from the air pump.
18	Brass Tee Fitting	A small fitting that connects the air pump's tubing to the suction cup line.

4.3 Final Major Component Selection Rationale

To make sure that all decisions made are as objective as possible, a systematic approach was taken to choose the major components in the system. This includes the microcontroller, the actuators, the onboard computer, and the camera. Every selection was based on a weighted criteria and a score from 1 (worst) to 5 (best). The scores are commutatively added and the choice with the highest score is the decision that team is going with.

4.3.1 Microcontroller Selection

A comparison was dawn between the Arduino UNO against two other common microcontrollers, those being the STM32 Nucleo and the ESP32. Both the boards competing with the Arduino possess faster speeds and memory; however, the nature of the project does not require such power. The tasks require are simple and can be handled by an Arduino. The most critical criteria was familiarity, availability, and experience. The group used Arduino IDE before and are well accustomed with how it operates. Furthermore, there are well-tested libraries/resources online. For these reasons the Arduino was selected [1], [2], [3].

Table 4-2: Weighted decision matrix - microcontroller selection.

Criteria	Weight	Arduino Uno	STM32 Nucleo	ESP32 Dev Board
Price (Arduino ~92 AED vs STM32 ~55–92 AED vs ESP32 ~22–37 AED)	0.20	4	3	5
Processing Speed	0.10	2	5	4
Matches 5 V Logic of Servo/Motor Drivers	0.20	5	3	2
Library and Community Support	0.20	5	3	4
Ease of Setup (no extra circuitry, familiar IDE)	0.20	5	2	3

Built-in Wireless (not needed. Use HC-05)	0.10	2	2	5
Weighted Score	**1.00**	**4.20**	**2.90**	**3.70**

4.3.2 Camera Selection

The camera options were explored for the computer vision subsystem: Logitech C270 webcam, Raspberry Pi Camera Module V2, and a generic InnoMaker USB camera. In theory, both options are superior; the Pi Camera and InnoMaker both offer 1080p while the InnoMaker camera has a much larger field of view. The biggest factor was that the team already had a Logitech C270 so using it meant both time and cost was saved. Not only is the C270 a standard USB camera that works with OpenCV on Windows and Linux without any special drivers, but it also solved the driver problems that were encountered when attempting to get the Pi Camera to work on Ubuntu. Its 720p video is more than adequate for the object detection model and the color detection used so the additional resolution and field of view of the other cameras wasn't worth the additional cost and setup time [4], [5], [6].

The camera options were explored for the computer vision subsystem: Logitech C270 webcam, Raspberry Pi Camera Module V2, and a generic InnoMaker USB camera. In theory, both options are superior; the Pi Camera and InnoMaker both offer 1080p while the InnoMaker camera has a much larger field of view. The biggest factor was that the team already had a Logitech C270 so using it meant both time and cost was saved. Not only is the C270 a standard USB camera that works with OpenCV on Windows and Linux without any special drivers, but it also solved the driver problems that were encountered when attempting to get the Pi Camera to work on Ubuntu. Its 720p video is more than adequate for the object detection model and the color detection used so the additional resolution and field of view of the other cameras wasn't worth the additional cost and setup time [4], [5], [6].

Table 4-3: Weighted decision matrix - camera selection.

Criteria	**Weight**	**Raspberry Pi Camera V2**	**InnoMaker USB Camera**	**Logitech C270**
Image Quality (5%)	0.05	4	4	3

Ease of Integration (25%)	0.25	2	4	5
Availability & Cost (40%)	0.40	1	1	5
Software Compatibility (25%)	0.25	2	4	5
Reliability (5%)	0.05	4	3	4
Total Score	**1.00**	**1.80**	**2.75**	**4.85**

4.3.3 Mini-Computer (Vision Process) Selection

Raspberry Pi 4 was compared with the NVIDIA Jetson Nano and the FriendlyElec NanoPi M4 for running the object-detection software. The built-in graphics chip makes the Jetson Nano's processor much faster at AI jobs, while the NanoPi M4 is also faster on paper. The Raspberry Pi 4, however, is about a third of the price of the Jetson Nano (around 128–202 AED versus 363 AED) and has Wi-Fi and Bluetooth built in, meaning that there was no need for extra wireless equipment, and there is much more documentation and example code available for running OpenCV and Flask on the RPi 4. The object-detection model was found to be fast enough to run comfortably on the Pi 4's processor at the speed of dashboard updates (250 ms), and the same was true of the color detection. The Jetson Nano's extra processing power wasn't needed for this application [7], [8], [9].

Raspberry Pi 4 was compared with the NVIDIA Jetson Nano and the FriendlyElec NanoPi M4 for running the object-detection software. The built-in graphics chip makes the Jetson Nano's processor much faster at AI jobs, while the NanoPi M4 is also faster on paper. The Raspberry Pi 4, however, is about a third of the price of the Jetson Nano (around 128–202 AED versus 363 AED) and has Wi-Fi and Bluetooth built in, meaning that there was no need for extra wireless equipment, and there is much more documentation and example code available for running OpenCV and Flask on the RPi 4. The object-detection model was found to be fast enough to run comfortably on the Pi 4's processor at the speed of dashboard updates (250 ms), and the same was true of the color detection. The Jetson Nano's extra processing power wasn't needed for this application [7], [8], [9].

Table 4-4: Weighted decision matrix - minicomputer selection.

Criteria	Weight	Raspberry Pi 4 (4 GB)	NVIDIA Jetson Nano (4 GB)	FriendlyElec NanoPi M4
Price (Pi 4 ~128–202 AED vs Jetson Nano ~363 AED vs NanoPi M4 ~184–275 AED)	0.25	5	1	2
Processing Power for Object Detection	0.20	3	5	4
Built-in Wi-Fi/Bluetooth	0.15	5	1	3
Documentation and Community Support	0.20	5	3	2
Pin Compatibility with Sensors/Modules	0.10	5	4	4
Power Use and Heat	0.10	5	2	3
Weighted Score	**1.00**	**4.60**	**2.60**	**2.85**

4.3.4 Arm Actuator

Three different types of motor were tested: servos (e.g. MG995, SG90), brushed DC motors, and stepper motors. The primary reason for selecting servos was because they already had a built-in position sensing system; a DC motor or stepper would have needed another sensor (an encoder) to do the same. Servos also share a single signal wire with the servo driver board already installed in the system, while DC motors have to have a separate motor driver circuit and stepper motors must have a dedicated driver chip. Servos were chosen for the wrist, elbow and gripper for this project because of the simplicity of wiring, the built-in position feedback, and a torque output that was more than enough for the arm, although they were not as precise

at positioning as stepper motors, and DC motors with gearboxes would be cheaper and still provide strong continuous torque [10], [11].

Table 4-5: Decision matrix - robotic arm actuator.

Criteria	**Weight**	**Servo (MG995/SG90)**	**DC Motor + Gearbox**	**Stepper Motor**
Knows Its Own Position (no extra sensor needed)	0.25	5	1	4
Simple Wiring (plugs into servo driver vs. needs its own driver circuit)	0.20	5	2	2
Torque for the Price (MG995: ~11 kg·cm at 6 V, ~29–51 AED)	0.20	4	4	3
Price per Unit (MG995 ~29–51 AED vs DC motor ~11–22 AED vs stepper ~37–73 AED)	0.15	3	5	2
Holds Its Position with No Power Applied	0.10	5	1	5
Speed of Movement	0.10	3	5	2
Weighted Score	**1.00**	**4.30**	**2.80**	**3.00**

4.4 MG995 Torque Sizing for the Arm

The wrist and elbow joints were chosen to use the stronger MG995 (rated around 3 kg·cm) servo instead of the lighter MG900 (rated around 1.8 kg·cm) as these two joints are the most loaded when the arm is fully stretched out to reach an object. The MG995 is rated at roughly 11 kg·cm of torque at 6 V, rising to about 13 kg·cm at 7.2 V . To prove this, a simple simulation of the fully extended arm was performed by adding the weight of the forearm, the gripper and a representative object (an egg, nut, bolt or ball). As seen in the figure below, the torque produced by the actuators is more than sufficient and can be used for safe operations.

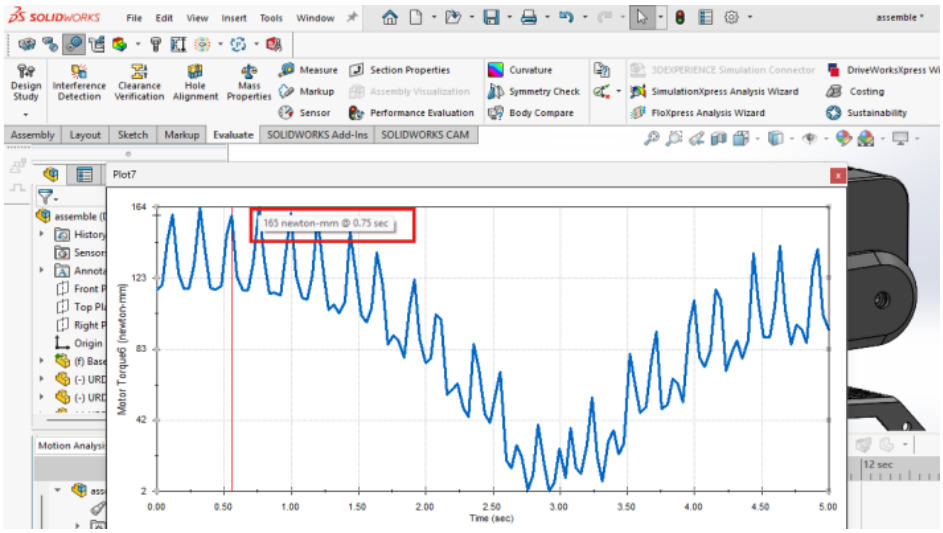

Figure 4-3: Simulations with a payload of 250g confirm the choice of the motors.

4.5 Power Supply Selection

Power is arranged in three voltage levels. The 12 V line is the raw battery line that is fed into a buck converter that reduces the voltage to 6 V. This 6 V rail is used to power the V+ pins of the PCA9685 servo driver, which then powers all three arm servos, and also powers the the DC air pump (via the relay) and the two wheel motors (via the L298N). The logic/signal line (5 V) provides the VCC of the Arduino, the Raspberry Pi, the Bluetooth module, the signal side of the PCA9685, and the signal/logic side of the L298N motor driver and the relay module. The 5 V logic rail is isolated from the 6 V/12 V power rail so that a sudden surge in current from the servos, motors or pump does not cause a voltage drop that could reset the Arduino or the Raspberry Pi. As illustrated in Figure 4-1, all of the grounds, including the 12 V battery, the 6 V buck converter output, the 5 V rail, the Arduino, the Pi, and all other modules, are connected together on one common ground bus. This prevents electrical noise from the high current wiring for the motors, pump, and servos from interfering with the signal wiring. The 12 V rail is powered directly by the main battery pack, which has a capacity of 12 V, 3000 mAh (36 Wh).

The switching buck converter is used rather than a linear regulator to step down the 12 V to the 6 V required by the servos, motors and pump which draw several amps at peak load, because the 12 V to 6 V conversion with a linear regulator would dissipate a significant amount of energy as heat. A switching buck converter will convert at greater than 85% efficiency and will not overheat like linear regulators do when operating under this type of load.

4.6 Power Budget

The current used by each component of the system is shown in table below at the voltage at which it operates. This is used to size the battery and to ensure the buck converter and motor driver don't have to deliver more current than they are rated for. The servo numbers are worst case (stall) numbers, as if all three servos are pushing against something at the same time, the DC motor numbers are typical running numbers, not stall numbers.

Table 4-6: Power budget summary.

Subsystem	Supply Voltage	Typical Current	Peak Current	Power (Peak)
Arduino Uno (logic)	5 V	50 mA	80 mA	0.40 W
HC-05 Bluetooth Module	5 V	30 mA	40 mA	0.20 W
PCA9685 Driver (logic/signal)	5 V	10 mA	15 mA	0.075 W
MG995 Servo (Wrist)	6 V (via PCA9685 V+)	170 mA	2.0 A	12.0 W
MG995 Servo (Elbow)	6 V (via PCA9685 V+)	170 mA	2.0 A	12.0 W
SG90 Servo (Gripper)	6 V (via PCA9685 V+)	100 mA	650 mA	3.9 W
L298N Motor Driver (logic)	5 V	36 mA	70 mA	0.35 W
DC Motor 1 (mobile base)	6 V (via L298N)	300 mA	1.2 A	7.2 W
DC Motor 2 (mobile base)	6 V (via L298N)	300 mA	1.2 A	7.2 W
DC Air Pump	6 V (via relay)	250 mA	400 mA	2.4 W

Subsystem	Supply Voltage	Typical Current	Peak Current	Power (Peak)
5 V Relay Module (coil)	5 V	70 mA	70 mA	0.35 W
Raspberry Pi 4	5 V	600 mA	1.2 A	6.0 W
Total 6 V Rail	6 V	1.29 A	7.45 A	44.7 W
Total 5 V Rail	5 V	0.80 A	1.48 A	7.4 W

If all three servos stall at the same time, and both wheel motors and the air pump are operating at full power, then the 6 V rail has to draw about 44.7 W from the 12 V battery, which is about 7.45 A, considering the efficiency of the buck converter (around 85%). This equates to a worst-case scenario of 41 minutes if this load were to be maintained continuously, but this is not likely to be the case in normal operation as servos and motors are not expected to stall simultaneously for long periods. The 6 V rail consumes approximately 7.74 W, which would represent approximately 0.76 A from the 12 V battery, resulting in an estimated run time of about 4 hours enough for development, testing and demonstration sessions. The 6 V rail peak current is 7.45 A, which is much larger than the average current (1.29 A), so the buck converter and wiring should be capable of handling at least 8 A.

4.7 Final Electrical Schematic

The figure below is an electrical schematic showing how different components are wired together, and what was connected to the individual ports of each component.

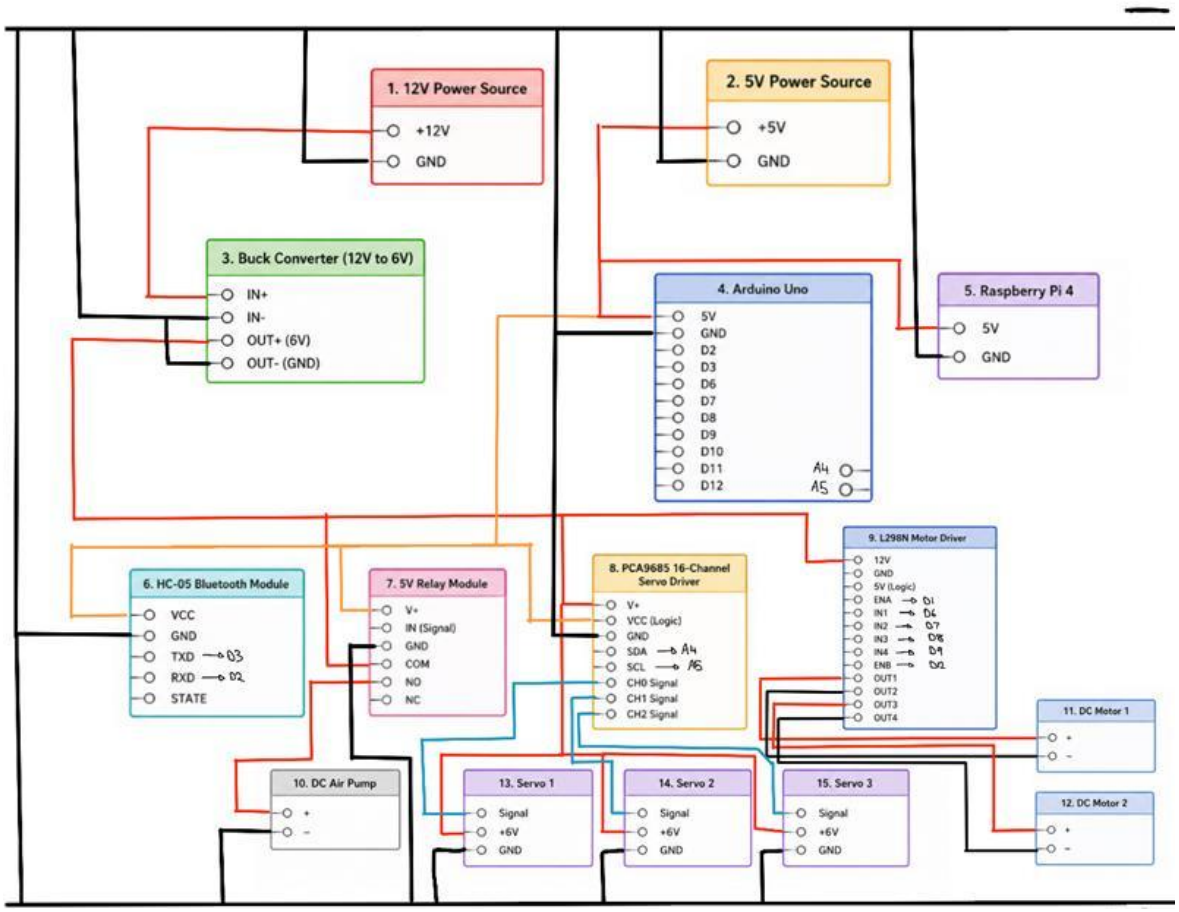

Figure 4-4: Electrical schematic of the system. Red shows 12-6 V power lines, black is for ground, blue is for servos signal lines, and orange is for 5 V VCC line.

5. Software Design Narrative

5.1 Software Design Rationale

5.2 Vision Detection System

A vision detection system was developed in order to provide real-time object recognition enabling tracking which is vital for the system to run autonomously. This system combines deep learning-based detection using YOLOv8 with image processing techniques utilized such as HSV color filtering to improve reliability under different conditions. The development process included dataset acquisition, image annotation, model training, sensitivity tuning, camera integration and the performance validation through a series of testing. The trained model was then deployed onto the Raspberry Pi 4 and integrated with the camera system to allow embedded operation without the need for external connections and computing resources. Additional testing was conducted to test detection accuracy, latency, connectivity and performance to understand how useful it was when tested under conditions similar to the final

test. The complete development process is discussed from the dataset preparation to model training and system testing.

5.2.1 Dataset Acquisition

As it is required to detect 5 objects such as an egg, nut, bolt, compass and ball, it is vital to obtain datasets which can later be used for the process involved in training the model.

The dataset consisting of egg images were obtained from the Open Images Dataset as it provides a large collection of images suitable for model training applications. This tool was used to identify and download images with a diverse set of samples with varying backgrounds, lighting conditions, object orientations and types of eggs. The way in which these were taken from the website was through the software Anaconda, the object class “egg” was specified as the search target with 100 specified as the amount of photos taken from the set. An organized folder consisting of these images was created and stored. Duplicated or invalid images were filtered out to improve overall dataset quality and error handling routines were included to remove corrupted image files. The compass detection was made of images that were taken manually under different lighting conditions and backgrounds, these were manually annotated and incorporated within the rest of the dataset.

For the nut and the bolt, these datasets were partially created manually and partially obtained from online resources. This is useful to ensure that images consisting of different lighting was used alongside having images with many bolts and nuts as opposed to just one, however the dataset was partially manually created as it allows the trained model to identify the specific model that will be used. The website Roboflow was used to obtain the dataset, and a similar process was used to filter images, unclear images or overcrowded images were removed and replaced with more reasonable images that will aid the dataset in identifying it. The following figure displays the datasets used for the egg and bolt detection.

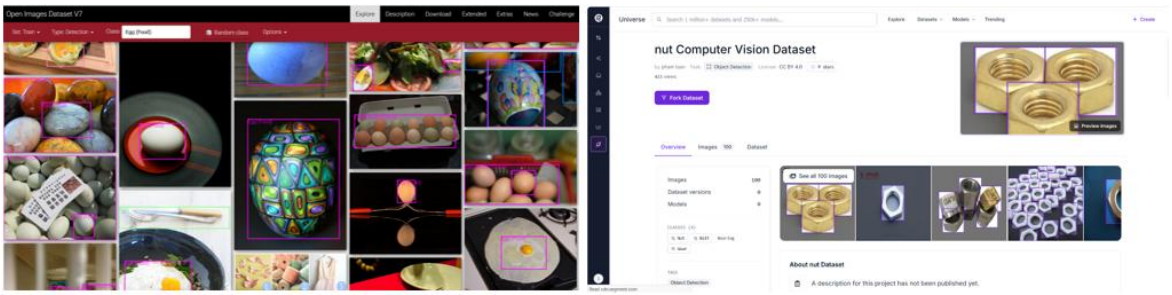

Figure 5-1: Egg (left) and Nut (right) Datasets used for Detection

The collected datasets provided a broad range of images related to each object enabling the model to learn distinguishing features over different viewing conditions and environments. Variation is highly crucial within the dataset for improving model generalization and decreasing the sensitivity to changes related to lighting, scale and orientation. The dataset formed a foundation for the training pipeline with high image quality and diversity playing an important role within the accuracy of the model. Dataset organization and management was carried out later to facilitate efficient annotation and validation, allowing for a smooth transition to the testing process.

5.2.2 Image Annotation and Labelling

During the process of training of the model, it is required to annotate the training data to allow the model to identify the target object and location within an image. Annotation is extremely crucial to expose the model to truth information which is vital for training and allowing the model to associate image features with an object class. Bounding boxes were used to define the position of each object allowing the model to learn classification and localisation tasks. Accurate annotations allow for achieving high detection performance whilst incorrect labels can negatively affect model accuracy and reliability.

The collected datasets were imported into CVAT, which is an open-source image annotation platform which is mostly used for computer vision applications. This resource was utilized due to its efficient annotation tools alongside its compatibility with YOLO training formats and support for object detection tasks. Bounding boxes were manually drawn around each target object as shown in the figure, the appropriate class label was then assigned. Separate classes were created for the eggs, nuts and bolts to enable multi class object detection during model training.

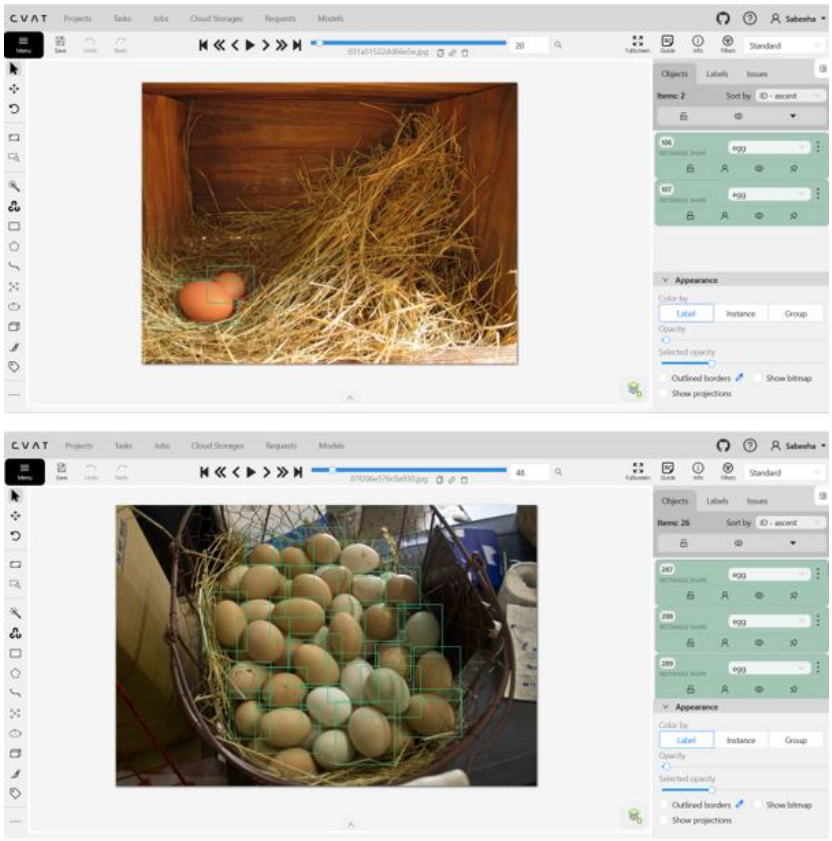

Figure 5-2: Bounding Box Annotations used for Training

Images were annotated systematically to maintain consistency throughout the dataset and labelling inconsistencies were investigated to ensure there was no incorrect assignment of the object. Once the annotations were complete, they were exported as a YOLO1.1 file so it can directly be used within the training pipeline. This organised annotation workflow improved dataset quality and reduced the likelihood of training errors during model development, the assignment of objects was as follows: (1) Egg (2) Nut (3) Bolt (4) Compass, these affected the label annotations as shown in the figure below. This figure displays an egg annotation; in the image there appears to be 5 eggs therefore there are 5 rows of coordinates showing the bounding box location in the corresponding image file. This ensures that the model understands which label corresponds to which object. It is also important to note that the txt file names correspond exactly to the image files.

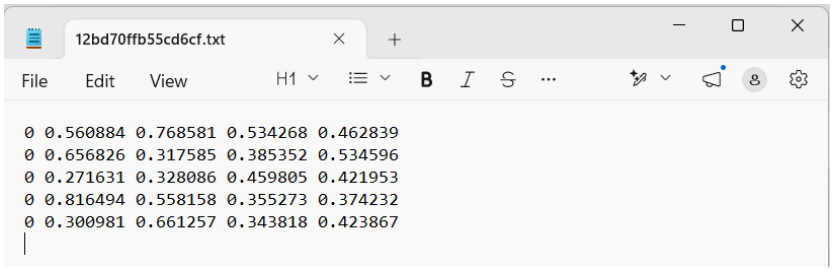

Figure 5-3: Txt File for Egg Detection (1)

Following the annotations of the dataset, it was partitioned into training and validation with 80% of the dataset used to train and 20% used to test the effectiveness of the model. This allows for model development alongside performance evaluation.

5.2.3 Training the YOLOv8 Model

Following the completion of the manual annotation process in CVAT, the labels were exported in the YOLO1.1 format during the model training. It was separated into training, validation and testing subsets to support the model development and performance evaluation. Splitting the dataset into these various categories ensures that the model learning, validation and final performance assessment were conducted using independent image sets. The dataset was also reviews to ensure that all the images corresponded to the appropriate file which reduces errors during testing.

A ‘dataset.yaml’ file was created to configure and define the locations of the training, validation and testing datasets alongside the names of the object classes within the project. This was therefore used as a reference document for the YOLOv8 model allowing the framework to locate images and annotations while correctly associating each label with its corresponding object class, this ensures seamless multi-class object detection. The Ultralytics library was installed within a Python environment to provide access to the YOLOv8 framework and its useful utilities such as training, validation and model export. The Ultralytics implementation was selected due to its simplified workflow, comprehensive documentation and its ability to efficiently train custom object detection without the requirement of extensive network development from scratch.

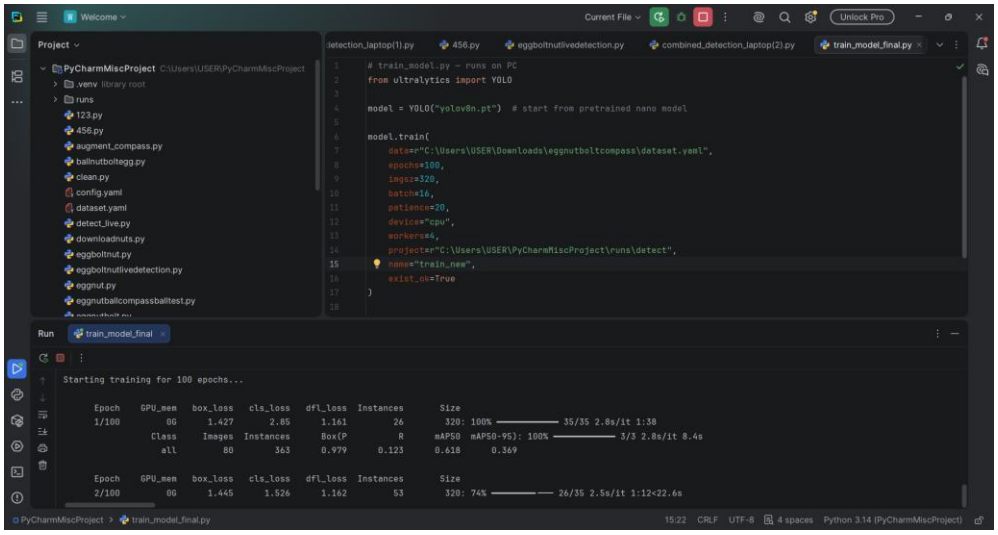

Figure 5-4: YOLOv8 Training using PyCharm in a Custom Dataset

PyCharm was used as the primary integrated development environment throughout the training process as shown in the figure above, this was primarily used for code development, model execution, debugging and performance monitoring. The training process enabled the neural network to learn the distinguishing visual features associated with each object class; the model analysed each image and attempted to identify the location and class of every annotated object. Model predictions were therefore continuously compared against the manually generated annotations to determine the magnitude of the prediction error. These errors were then used to update the neural network wrights through backpropagation which gradually improves the model performance over time. A training duration of 100 epochs was selected for the final model development performance, one epoch represents a complete pass of the entire training dataset through the network. The reason as to why 100 epochs was chosen was because it ensured the model was exposed to the complete dataset multiple times so it can progressively improve its understanding of object characteristics and features. Through trial and error, it was determined that increasing the number of epochs improved learning performance by providing additional opportunities for weight optimisation and error reduction, on the other hand excessively high epoch values can increase the risk of overfitting.

The key evaluation metrics included were precision, recall and the mean average precision which provides quantitative measures of object detection performance. Upon completion of training the model, a ‘best.pt’ model was used for real-time object detection, hardware integration and deployment testing on the Raspberry Pi platform. This combinations of a structured dataset, accurate annotations, appropriate training parameters alongside repeated

validation contributed to the development of a detection system capable of identifying objects under different conditions.

5.2.4 HSV Based Object Detection

In addition to the YOLOv8 model, HSV (Hue, Saturation, Value) colour filtering was implemented, this was done to detect and track the target green ball. This colour space was selected as opposed to the RGB colour model as it separates colour information from the image brightness, this in turn improves the detection under varying lighting conditions.

The HSV method identifies objects by filtering pixels that fall within a colour range that is predefined and it rejects pixels outside the selected thresholds. This process simplifies the image and therefore reduces background noise allowing for the target object to be isolated more effectively. OpenCV was used to convert images captured by the camera from BGR to the HSV format before the colour thresholds were applied.

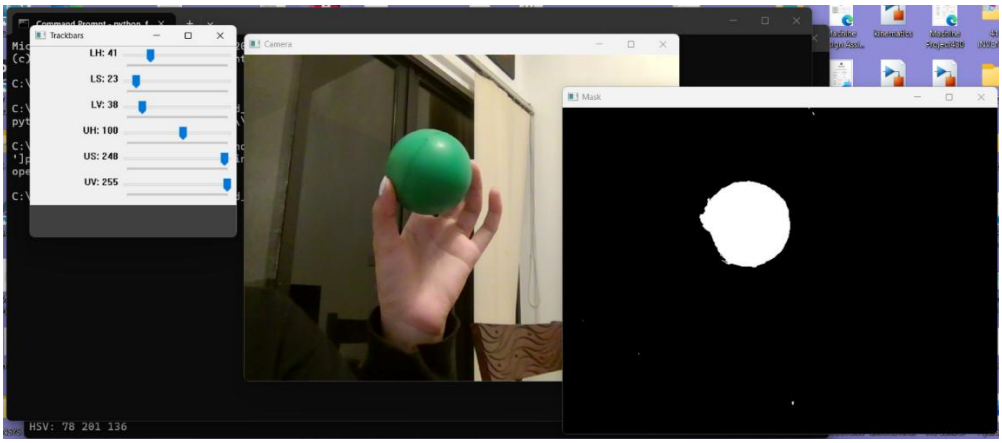

Figure 5-5: HSV Calibration Tool for Green Object Detection

A dedicated HSV script was developed to determine suitable threshold values for the green object, this allowed for interactive trackbars to change the following parameters:

- Lower Hue (LH): Minimum colour threshold
- Lower saturation (LS): Minimum colour intensity
- Lower Value (LV): Minimum brightness level
- Upper Hue (UH): Maximum colour threshold
- Upper Saturation (US): Maximum colour intensity
- Upper Value (UV): Maximum brightness level

A live camera footage was displayed as shown in the figure with the generated binary mask enabling immediate visual assessment of the segmentation performance. These threshold

values were altered iteratively until the green ball was accurately isolated from the surrounding background, in this way unwanted detections were therefore minimised. The resulting mask converted the target object into a white region and suppressed non-target areas as black pixels allowing the subsequent image processing operations to be more simplified. The selected HSV thresholds were chosen based on the ability to identify the target object consistently across multiple viewing angles and lighting conditions. Once suitable and tested HSV values had been determined, the calibrated thresholds were incorporated within the main vision detection program, this enabled them to generate binary masks during real time operation allowing for a more rapid identification of the target object's position within the camera frame. This detection provided a computationally efficient solution that could operate with minimal processing overhead on the Pi platform.

Integrating HSV filtering alongside the YOLOv8 model created a hybrid vision system that combined the accuracy of machine learning based object detection with the speed and simplicity of colour-based image processing forming a highly accurate and technical classification and identification system, vital for this project. The calibration process ensured that the selected thresholds were optimised for the project's operating environment therefore improving the overall detection reliability during testing and deployment.

5.2.5 Integration and Deployment

Once the model had been trained successfully with the 'best.pt' file generated, it was used to perform real-time object detection within a desktop development environment before it was then moved to embedded hardware. PyCharm was used as the primary development platform for testing and validating the object detection pipeline, initial testing verified that the trained YOLOv8 model could successfully load the generated model weights and perform live object detection using the camera embedded within the computer.

These tests allowed for the validation of model accurate, confidence values and object localisation performance before it was transferred to the system with the Raspberry Pi. Real time testing also provided for an opportunity to identify software related issues and optimise detection parameters prior to hardware integration.

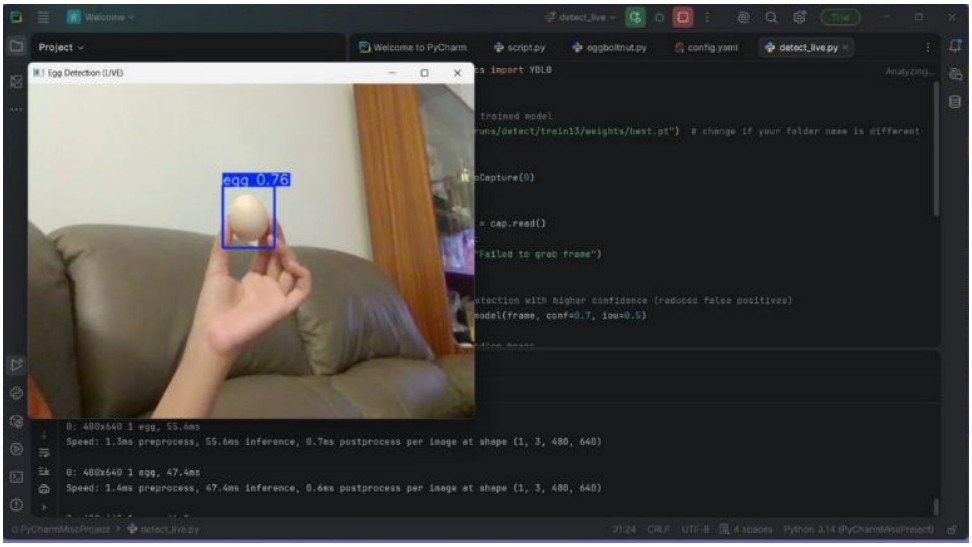

Figure 5-6: Successful Detection During Live Camera Testing of the Trained Model

This figure demonstrates a successful execution of the trained YOLOv8 model within the PyCharm environment, the displayed confidence score indicates the model's certainty in the detected object classification. The successful detection confirmed that the dataset preparation, annotation process and training procedure has produced a functional object detection model that can be deployed.

Detection confidence thresholds were adjusted to optimize object recognition performance; it acts as a sensitivity value that determines the minimum confidence required before an object is accepted as a valid detection. Lower confidence thresholds increased detection frequency but introduced a greater number of false positive detections however higher thresholds reduce false detections but resulted in missing the object on occasion. Multiple confidence values were evaluated during testing to determine the appropriate balance between detection accuracy and reliability. Sensitivity tuning was performed prior to deployment to improve the overall system, following a successful desktop testing and a trained 'best.pt' file, the model was transferred to the Raspberry Pi 4.

The Pi 4 was selected due to its more compact size, low power consumption alongside its suitability for computer vision applications. The model and associated required Python scripts were transferred from the computer to the Raspberry Pi using PowerShell and network-based file transfer methods. Ubuntu was installed on the Pi to provide a stable operating environment and the required software packages and dependencies were installed; this included the following:

- Python: Application development language
- Ultralytics YOLOv8: Object detection framework

- OpenCV: Image processing and camera interface library
- NumPy: Numerical and array operations
- PyYAML: Dataset configuration file handling
- Libcamera: Raspberry Pi camera management framework
- Logitech USB Camera Drivers: External camera communication

These installations ensured compatibility between the camera hardware, detection scripts and the trained model, configuration files and supporting project files were transferred in order to maintain consistency between environments. A Logitech USB Camera was integrated to provide a dedicated imaging solution for real-time operation, this enabled the camera connectivity, frame capture and live video streaming to be assessed and verified before moving forward. Image resolution settings were evaluated and improved to balance detection accuracy and processing speed, higher image resolutions improved visual detail but increased computational load whilst lower image resolutions improved frame rates but reduced available image detail. This meant that a suitable resolution was selected to achieve a more reliable detection performance whilst maintaining real-time operation on the Raspberry Pi.

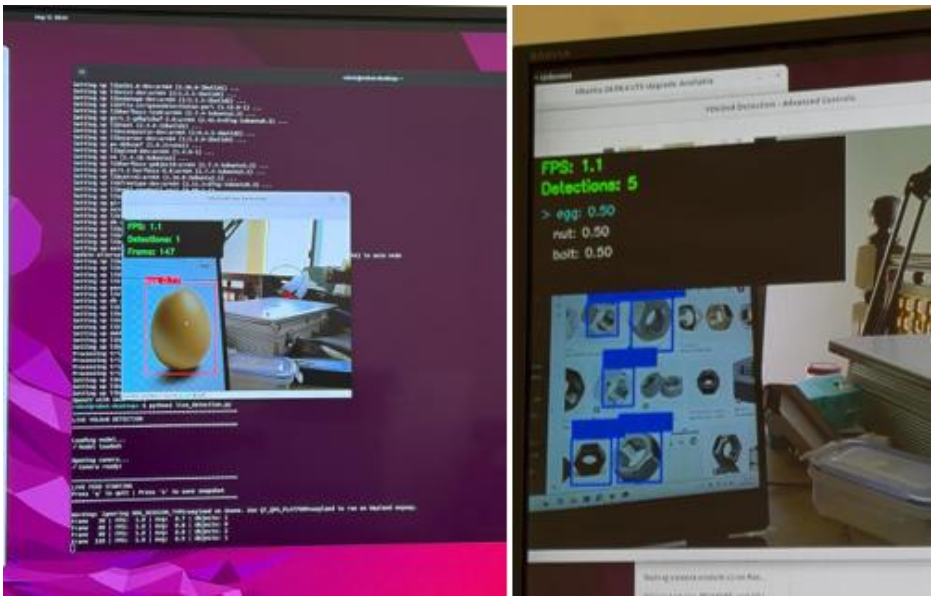

Figure 5-7: YOLOv8 Model Deployed on the Raspberry Pi

This figure demonstrates a successful integration of the trained YOLOv8 model onto the Raspberry Pi platform, the image on the left shows execution of the object detection software within the Pi environment following the installation of the required dependencies and transfer of the trained model. The image on the right demonstrates the system's ability to perform real-time multi-class detection. The detection confidence values and object counts are displayed

during operation providing feedback regarding the model performance and classification certainty.

Real-time testing was conducted to verify system performance under representative operating conditions, the detection accuracy, object localization performance alongside the system responsiveness were assessed using live camera feeds. Additional testing was performed to evaluate the stability of the camera connection and identify potential latency or communication issues. Successful operation across multiple object classes demonstrated the effectiveness of the training process and deployment, the final system provided reliable real-time object detection capabilities while operating entirely on the Raspberry Pi without the requirement for external computational resources. The combination of confidence threshold optimization, camera integration, software deployment and embedded processing resulted in a functional vision detection system.

5.2.6 Shape Detection

The vision system has been designed to identify and follow various objects in the robot workspace. There are several objects in the complete project (egg, nut, screw, compass, green ball), but for the purposes of this section, the green ball and compass will be taken as representative objects. The green ball was detected using a classical image-processing method, while the compass was detected using a YOLO-based detection method with the geometric validation.

A colour- and shape-based image processing technique detected the green ball. First, every frame of the video was converted from BGR to HSV colourspace. The HSV was chosen as it differentiated the colour information from the brightness, which made it more suitable to detect coloured objects under varying light conditions. To form the green color mask, the HSV range of green was predefined as [32, 50, 50] to [95, 255, 255]. This mask identified the pixels that probably came from the green ball.

Further filtering was added to the mask to improve the reliability. In order to ensure that the green channel was not weaker than the red and blue channels, a green dominance filter was used to check the green channel, to remove white, grey or dark regions that may be false ones to pass the HSV threshold. Bright low saturation pixels due to reflection were also suppressed by glare suppression. A region of interest was also created in the workspace to ensure that objects outside the robot workspace region were not considered. Finally, morphological opening and closing were carried out to minimize the small noises and fill small gaps in the mask.

The extracted contours in the binary image were obtained after cleaning the mask. For each contour, it was assumed that it was a ball. Multiple geometric filters were then used to identify if the contour was the correct shape of a ball. A minimum area filter was applied to remove very small contours. The circularity of each contour was calculated using:

$$\text{Circularity} = \frac{4\pi A}{P^2}$$

A is the area of the contour and P is the contour perimeter. The closer to 1 the better the shape is circular. Those that fall short of the circularity standard were disqualified. The aspect ratio of the bounding rectangle was also evaluated to make sure that the shape detected was neither too narrow nor too wide.

The minimum enclosing circle was computed for each of the accepted contours. This gave the center position of the ball and the radius of the ball in pixels. The fill ratio was determined using the ratio of area of the contour to the area of the enclosing circle. This confirmed the presence of a whole circular-shaped ball as the object detected rather than a partial and/or an irregular green shape. The code also calculated the distance to the ball from the camera based on the known ball diameter and the ball diameter measured in pixels:

$$\text{Distance} = \frac{\text{Real Diameter} \times \text{Focal Value}}{\text{Pixel Diameter}}$$

The candidates who estimated distance unrealistically were rejected. A score of area, circularity and fill ratio were used to select the final ball candidate.

The ball detection was then stabilized with a simple tracking system. The tracker saved the old ball's location, radius, distance, number of stable frames and number of lost frames. If the ball moved too far from the previous position, it was rejected if there were a sudden jump. The movement was smoothed by using weighted average, thus minimizing the jitter in the position displayed. The ball was only deemed “stable” when it was detected in multiple frames.

Not only did the trained YOLO model detect the compass, but it was also able to detect the compass using color thresholding. This was appropriate given that the compass looks more like a complex pattern than the green ball and couldn't be effectively identified with a simple color mask. The YOLO model was trained to detect object classes such as egg and compass, and the code explicitly picked out detections of the class name "compass". The model was able to generate bounding boxes, and confidence scores and class labels to be used for objects in the camera frame.

YOLO produced the candidate detections to which the code applied validation for the compass. Useful geometric information for each bounding box was generated: box width, box height, location of box center, and an estimated box pixel diameter. The pixel diameter was taken to be the average of the width and the height of the bounding box:

$$\text{Pixel Diameter} = \frac{\text{Box Width} + \text{Box Height}}{2}$$

This was used due to the fact that the compass is roughly circular in shape, and the apparent size of the compass in the image can be described with an estimated diameter.

The compass candidate was then tested with a number of filters. The first is that the confidence score for the YOLO method needed to be greater than the set confidence threshold for the individual compass. After that, the diameter of the pixel needed to be in a reasonable range between a minimum and maximum value. The aspect ratio of the Bounding Box was also validated, discarding any Boxes that were too tall, too narrow, or too broad to be considered a compass. A workspace region filter was provided as an optional filter that can be used to ignore detections made outside of the workspace area of the robot, if necessary. For each valid compass candidate the one with the highest confidence was chosen.

The compass distance was measured in the same way as the ball distance, by taking advantage of the same pinhole-camera effect. The diameter of the real compass was known and the distance can be estimated from the diameter of the image seen on the pixels and the calibrated focal length:

$$\text{Distance}_{mm} = \frac{\text{Real Compass Diameter}_{mm} \times \text{Focal Length}_{px}}{\text{Pixel Diameter}_{px}}$$

The diameter of the real compass was fixed at 60 mm, and the value of focal length may be entered manually or determined from calibration samples in the code.

The compass detection was implemented in a simple memory system to increase the stability. If the compass was lost for a short time, then the system didn't report it to be lost, but instead held the compass angle from the previous frame. The compass was considered to be stable only once it was detected for a desired number of frames. This helped to minimize flickering due to the occasional missed detection.

5.2.7 Distance Vision

An extension to the object detection and tracking system was the addition of distance estimation. The initial system was intended for multiple object detection and tracking: the green ball, compass, nut, screw and egg. The detection stage detects the object in the frame and the tracking stage tracks the detected object across successive frames. The distance estimation stage provides approximate depth information by estimating the distance between the camera and the detected object.

Distance estimation method relies on the correlation between the actual size and apparent size of the object in the image. The nearer the object to the camera, the bigger it will appear in the picture. The further it is from the camera, the smaller it will be seen. Hence, if the size of the object can be determined in pixels and a true size of the object is known, the distance between the camera and the object can be estimated.

The general Distance equation in the system is:

$$D = \frac{W_{real} \times F}{W_{pixel}}$$

The value of W_{pixel} depends on the object shape and the detection method used. For circular objects, such as the green ball, the pixel diameter can be calculated from the detected radius:

$$W_{pixel} = 2r$$

r is the radius of the detected object (in pixels). If an object is detected by the use of the bounding box (e.g., compass, nut, screw, egg, etc.), the size of the object can be determined by using the proper size of the bounding box that is suitable for the shape and orientation of the object such as width, height, average width and height, longest side, etc.

The green ball example is an application that uses HSV colour segmentation to isolate the green object from the background. Colour filtering, Suppression of Glare, Region-of-Interest filtering and Morphological operations were used to enhance the mask. Once the shape of the ball was detected, the system then performed shape-based checks, including circularity, radius, fill ratio, and aspect ratio, to determine if the object was a valid ball or not. The minimum enclosing circle was then used to obtain the ball centre and radius. The radius was converted to the diameter of the pixels and the distance was estimated. The ball tracker also smoothed and checked for stability to minimise sudden fluctuations and ensure a more reliable distance to be displayed.

The object here was detected by a model based on YOLO. The bounding box detected gave us the location, centre of object, width and height of the object. The compass was thin in the camera view and the pixel size was therefore approximated using the size of the bounding box. This pixel size was then used in combination with the size of the real object to determine the approximate distance between the camera and the object. This was also achieved by a simple memory-based tracking method where the system remembered the last valid compass detection for a short number of frames when the detector did not see the object for a while. This kept the output from the camera display stable.

The same distance estimation idea can be used for the entire target objects system. A dimension in the real world must be appropriate for each object. The ball is circular, therefore the diameter is used for the green ball. The outer diameter (or the bounding box width) of the nut may be used. The total length or the head diameter will be used for the screw depending on the camera view. The width or diameter of the compass can be known. The major axis or approximate height of the egg can be used. The dimension chosen should be the same as the pixel size measurement of the image.

Distance tracking is done by computing the approximate distance for each frame since an object has been detected and tracked. The size of the pixel being measured varies as the object moves, thus the estimated distance also varies. If the pixel size grows, the object is closer to the camera and estimated distance will get closer. In the situation where the pixel size decreases as time goes on, the distance is estimated as larger since the object is moving farther away. The change in distance over successive frames can be used to estimate the amount of translation in depth of the object.

The system's output shows the object's label, position indicator, object shape, estimated distance and tracking status. The “STABLE” label means that the object has been detected in several frames. This is significant because the distance can be influenced by detection inaccuracies, reflections, partial object visibility, and noise in a single frame. This is because stable tracking makes the displayed value more reliable, since the reliability of the display object is ensured by the stable tracking.

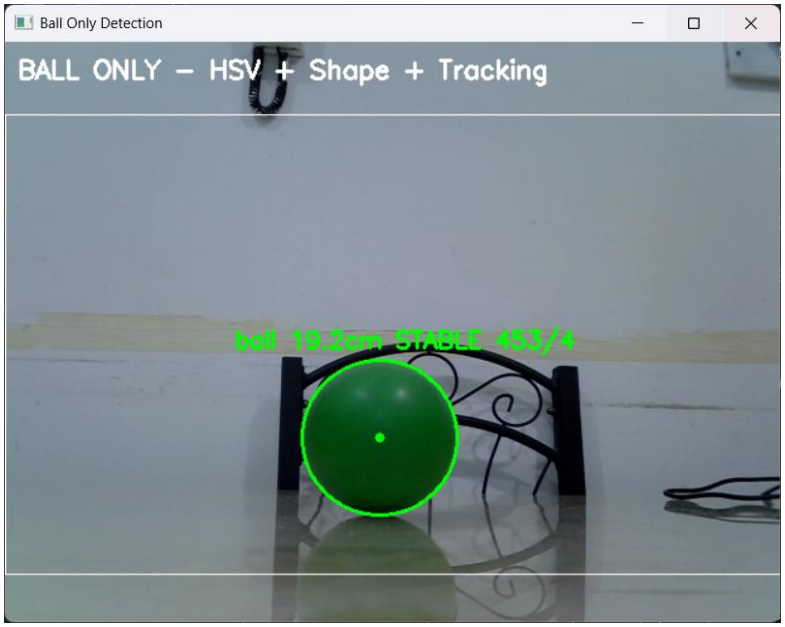

Figure 5-8: Green ball detection with distance estimation.

Figure 5-9: Compass detection with distance estimation.

Camera calibration is a critical part of the distance estimation accuracy. The values of the focal calibration are not standard, and vary depending on the camera used, the resolution, the position of the lens, the orientation of the objects, and the experimental set-up. The distance values shown should then be regarded as approximate values if they are not calibrated and verified with known reference distances. Objects can be calibrated by placing each object at a known distance from the camera and measuring the size of the pixels for each object, then calculating the focal calibration value using:

$$F = \frac{D_{known} \times W_{pixel}}{W_{real}}$$

The distance between the reference and the unknown points is D_known. It is best to repeat this at a few known distances in order to increase the confidence in the calibration. Final validation can be done by comparing the estimated distance with the measured real distances.

5.3 Web-Based Robot Controller Interface

The Pick and Place Robot Controller was designed as the primary control interface to the robotic system. The intent of this controller is to enable the user to control the robotic arm, mobile platform, gripping mechanism, camera stream and safety features from a single, well organized controller. The controller has buttons, sliders, switches, status indicators and command feedback instead of having to manually type commands in the Arduino IDE.

The controller is the link between the human and the robot hardware and is the Human–Machine Interface. By this interface, the user is able to choose the method for communicating, manipulate arm joints, move the mobile platform, control the gripper/pneumatic suction cup, watch the camera stream, and enable the emergency stop function, if necessary.

The first page of the controller is the entry page as shown in Figure 1. This page is meant to offer the user with an overview of the project and a starting point before proceeding to the main dashboard. It also enables the user to choose the communication manner. The controller can be connected using Bluetooth, USB Serial, Simulation Mode or Autonomous Network. This flexibility allows the controller to be used to manage real hardware, test without hardware, and more later, for future autonomous operation using the Raspberry Pi.

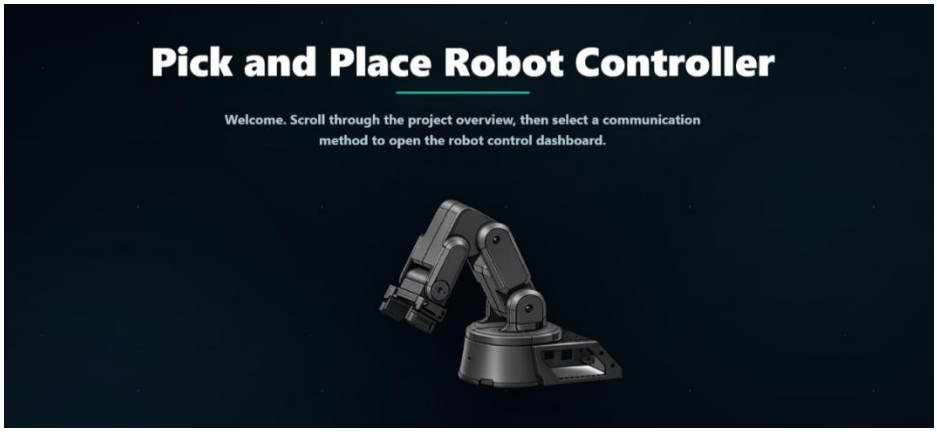

Figure 5-10: Entry page of the Pick and Place Robot Controller.

The main dashboard is centered around the Robot Operation Center, shown in Figure 2. The connection status, selected communication method, Arm Mode, Platform Mode and the safety-related controls are located in this section. Use of operating modes is used to organise the system and lead the user to only show the controls they don't need to see at once. When the

arm is in Manual Mode, the arm sliders will be displayed. The WASD movement controls are displayed when the platform is in Manual Mode. This will make the controller more easy to grasp and easier to operate.

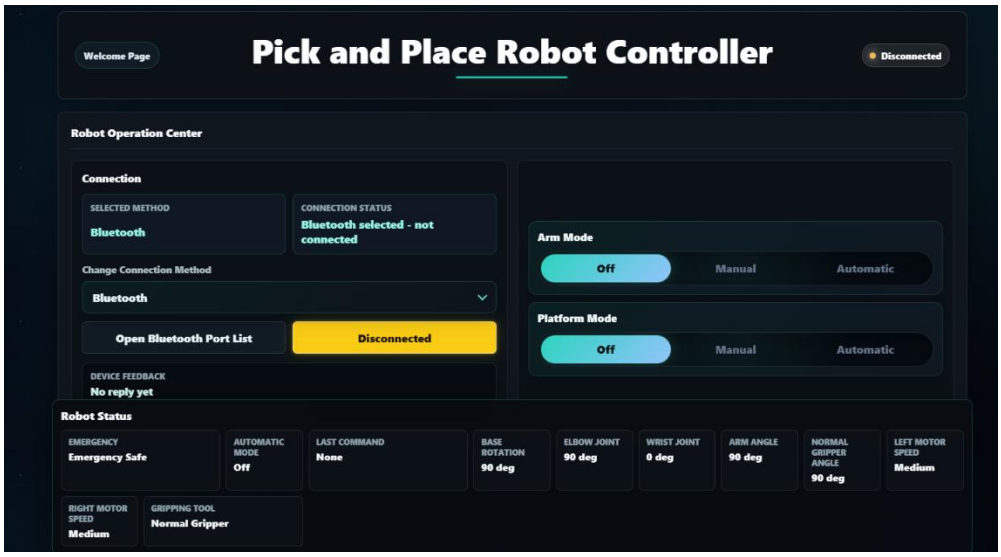

Figure 5-11: Robot Operation Center showing connection state, operation modes, and safety controls.

Command-oriented connectivity is a fundamental notion in this control system. The control web cannot move the motors on its own. In other words, it transmits understandable instructions to either the Arduino or the Raspberry Pi bridges. Examples are: ELBOW_JOINT:90, WRIST_JOINT:0, NORMAL_GRIPPER_ANGLE:90, SUCTION_CUP_ON, PLATFORM_FORWARD, and EMERGENCY_STOP. This command format facilitates debugging since the identical instructions may be seen in the command log and confirmed via the Arduino Serial Monitor.

The controller operates via Bluetooth and USB with the Web Serial API. This way the browser can talk to other serial equipment like an Arduino (either via the USB or a Bluetooth device or connector that is visible as a serial COM Port). This can be useful for the controller, since it can initiate commands directly from the browser without requiring any additional desktop software. The newer version also has an Autonomous Network option, which means that the controller can be used to send commands via a command bridge running on a Raspberry Pi via an HTTP request. This will enable autonomous operation in Wi-Fi mode in the future.

The manual control part of the dashboard is shown in Figure 3. The arm control section controls the robot joints with sliders and the platform control section has a WASD type layout. Joint control is easier with the sliders, since the user doesn't have to type commands to select a value. The platform controls feel familiar as WASD often equates with movement control. The

controller also provides the ability to select the speed of the platform motors including slow, medium and fast mode.

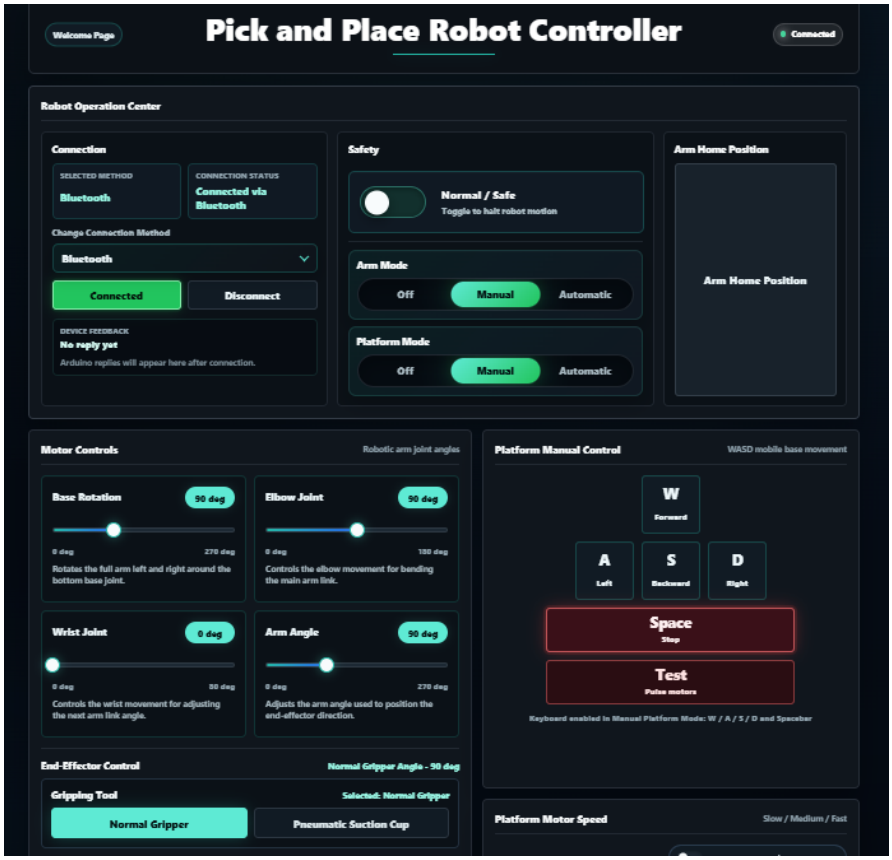

Figure 5-12: Manual Arm and Manual Platform control panels.

An additional key idea in the controller is the calibration of the user interface and the actual hardware. It may be possible for the value displayed to the user to not always be the same raw value used by the motor driver. For instance, the range on the wrist slider might be a clean range for the user, but the range might be a safe range for PCA9685 on the Arduino. This ensures that the robot doesn't hit its mechanical boundaries and also makes the interface user-friendly. This is particularly crucial as the safe ranges of servo motors, linkages, and grippers could vary after assembly.

The controller also features a camera block, command log and robot status panel as seen in Figure 4. The camera block is for camera viewing and ready to be streamed using Raspberry Pi. The controller can receive a raw Raspberry Pi stream, and is ready to receive a future stream from a Raspberry Pi, such as a YOLO/object-detection stream. The Browser Camera feature is designed to enable the browser to access available cameras and media input devices. However, the controller itself does not run the object detection model. Rather, the controller is supposed to display the stream or the output of the model, which is run on the Raspberry or laptop.

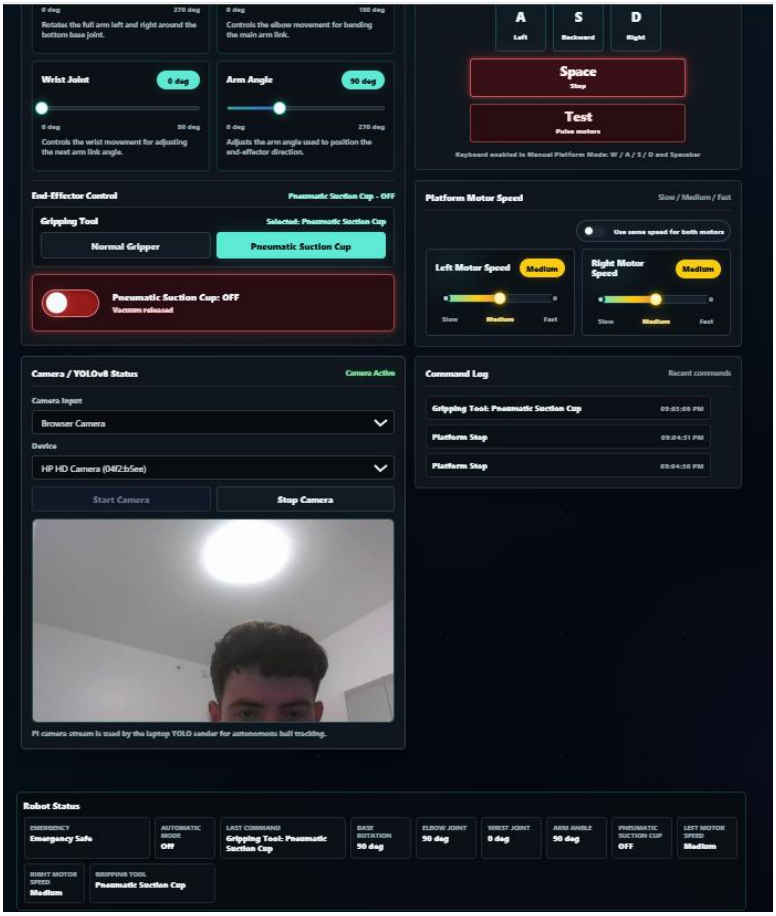

Figure 5-13: Camera stream, command log, and robot status panel.

Both the command log accompanying robot information display are vital for providing operational response. The command log displays whatever the controller delivered, but the machine's status screen displays the system's present condition, including emergency mode, previous command, motor orientation, gripper condition, and platform pace. This is consistent with the usability concept known as visibility of system status, which states that the interface should keep the user informed about what is going on and whether instructions are currently being processed.

Safety is also an important issue in this microcontroller. The Emergency Mode was designed with the intention of halting the robot’s outputting as well as prohibiting further movement orders whereas emergency mode is engaged. This involves preventing arm motion, platform motion, plus the pneumatic pump. The controller also employs disconnected-state safety, which means that if the system is disengaged, motion modes must be turned off and no orders should be issued. This is significant since the robot shouldn't continue to function if communication is lost.

6. Mechanical Design Narrative

The mechanical structure of the robot is comprised of two subsystems fabricated by two different processes, each selected according to the specific requirements. The 4-DOF arm consists of all 3D-printed parts made from PLA material, and the complex joint geometry, servo mounting features and the gripper mechanism are all produced and iterated without the need for tools. The mobile base chassis is a single acrylic laser-cut plate that is selected for its rigidity and flatness to ensure alignment of the two drive motors and a stable mount for the arm, electronics and battery, which can be easily modified.

6.1 Arm Structure

The arm's kinematic chain is defined in the URDF as base_link -> elbow_link -> wrist_link -> arm_link -> end_link. The servos used are an MG995 or SG90, each of which is mounted in a printed bracket on one link, and a circular servo horn is bolted to the next link in the chain, allowing the servo body to rotate the servo horn directly.

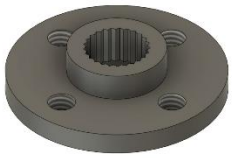

Figure 6-1: Circular servo horn used for the joint fitting.

Notably, the original design was such that in addition to the elbow, wrist and arm joints, which all rotate about the horizontal (Y) axis, the shoulder joint was designed to rotate the entire arm about the vertical (Z) axis. This shoulder joint was determined to be redundant as part of the mechanical design process: The mobile base is already capable of yaw rotation of the entire robot with its two independently driven DC motors, so rotating the arm about Z can be done by steering the base. The shoulder joint is therefore operated by rotating the platform as opposed to the shoulder joint itself. Though this reduced the DOF of the arm by one, the system lost a redundant joint, which reduced weight and power demands. This is not a restriction, it's merely a design simplification, as there are less parts, less weight and fewer servos needed, but the chassis' drivetrain delivers the same functionality.

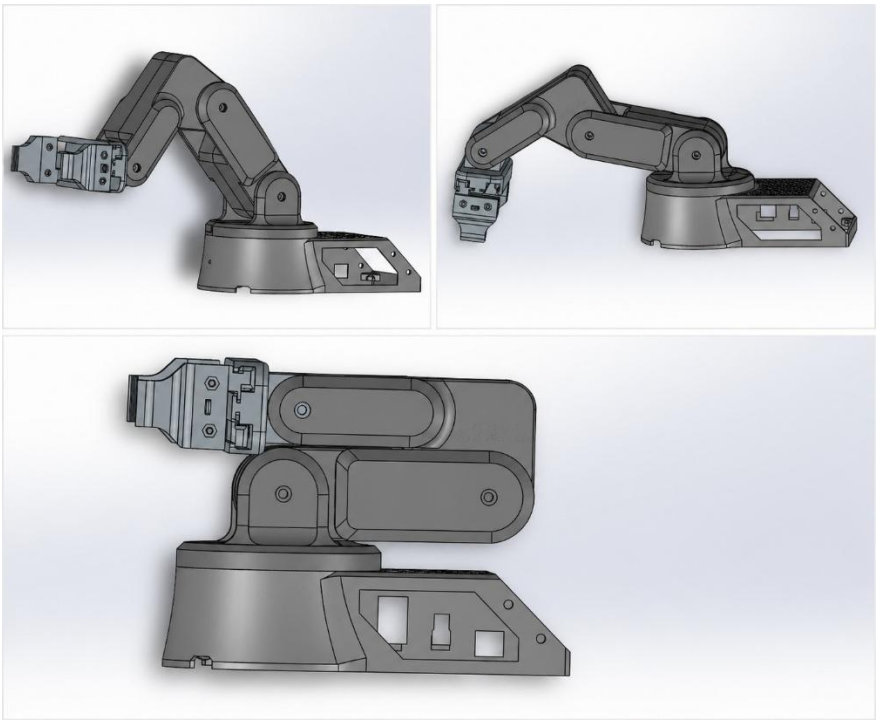

Figure 6-2: 3D CAD render of the robotic arm in SolidWorks.

The end effector is a 3D-printed rack and pinion gripper. Like the other joints of the arm, a pinion gear is attached directly to the output shaft of an SG90 servo, and it is attached to two linear racks, which are driven symmetrically to open and close the gripper's jaws. A suction cup is attached to the top of the gripper casing, which can be used to grip objects that are not easily grasped by the jaws.

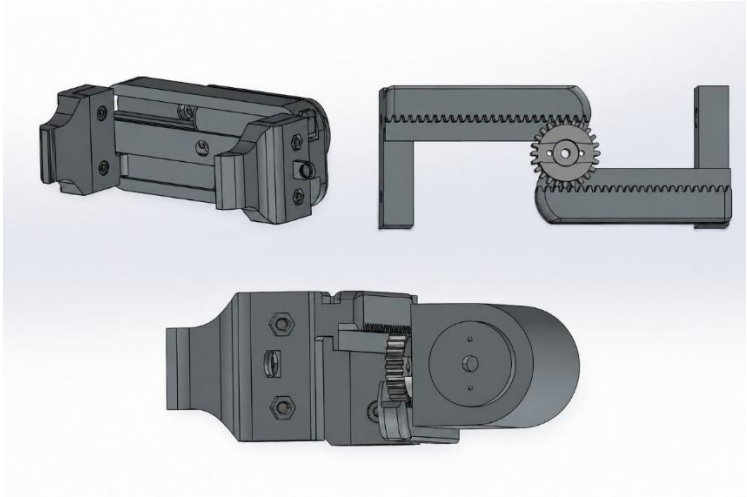

Figure 6-3: 3D CAD render of the gripper mechanism.

The entire arm and gripper assembly was printed with PLA, as it was found to be cheap, reliable and had good dimensional accuracy, with the servo pockets and horn-mounting features being printed accurately at each of the servo joints, and the material being sufficiently stiff and strong for the loads involved, consistent with the torque margins established earlier for the MG995 servos. It was also cheap enough and quick to print that it was feasible to make a few changes to each link (e.g. eliminate the shoulder joint from the design) without too much cost or delay.

Figure 6-4: Physical assembly of robotic arm.

6.2 Base Platform

The mobile base is based on a single circular acrylic plate, laser-cut as a flat structural platform for the whole mobile system, 30 cm in diameter. The two DC drive motors are located in the center of the plate, creating differential-drive locomotion and steering for the robot. No printed adapter is needed between the arm's base link and the acrylic at the front edge of the plate. There is a large unoccupied portion of the plate for the mounting of the electronics (Arduino, motor driver, servo driver, Bluetooth module) and the battery. The chassis was built from acrylic, which can be laser-cut quickly and accurately, and the resulting flat, rigid plate ensures that the two drive motors are aligned relative to each other (which is crucial for accurate differential-drive motion), and also allows mounting holes and cutouts for the drive motors, arm base, and electronics to be positioned accurately and changed if necessary should the layout change.

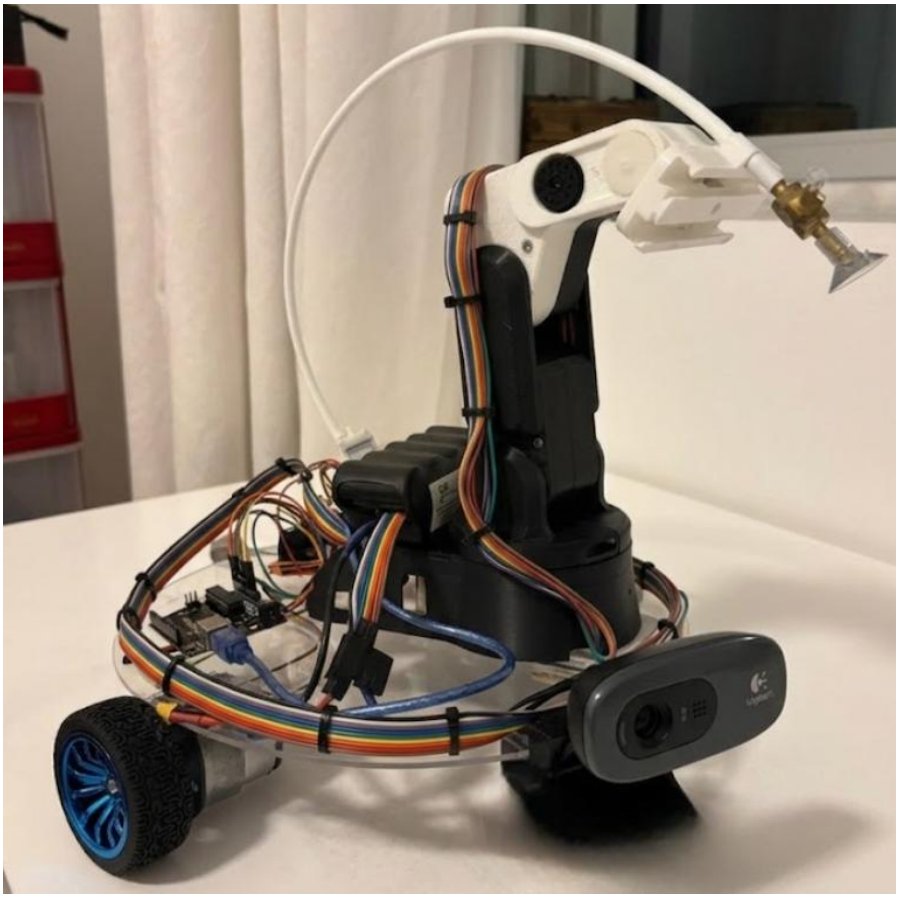

Figure 6-5: Fully assembly robotic arm attached to the platform.

6.3 Dimensions and Workspace Envelope

When the arm is not in use, it folds over the robot chassis, and the robot's footprint is just the 30cm circular acrylic plate. The arm length (from the plate to the end of the arm) is about 16.5 cm at the furthest and lowest position. From the base of the arm to the lowest position of reach (including the length of the suction cup attached to the gripper) is about 22 cm. When the height of the end effector is not limited, the maximum horizontal distance of the arm can be extended to about 26 cm. The difference is a result of the geometry of the arm: to reach the furthest horizontally, the end effector must be brought up a little, and to reach the lowest point, the arm must be folded into a more compact shape, which decreases the horizontal reach. These numbers are not to be confused with the workspace values that were reported in the inverse kinematics section (Table 7-1), where a set of points in X = 0.05–0.20 m, Y = −0.10–0.10 m, and Z = 0.02–0.07 m were found to be reachable by the IK solver. The values were just test points to ensure that the solver was working properly, and not a complete description of the actual range of the arm. These figures of 22 cm and 26 cm are the maximum reach measured with the arm including the suction cup and are to be used as the actual reach envelope of the arm.

Table 6-1: Robot footprint and arm reach envelope.

Measurement	Value	Reference Point
Chassis diameter (home position footprint)	30 cm	Circular acrylic plate
Extended reach beyond plate edge (furthest/lowest point)	~16.5 cm	Edge of plate
Maximum reach to lowest point (incl. suction cup)	~22 cm	Arm base
Maximum horizontal reach (height unconstrained)	~26 cm	Arm base
IK solver sample range — X (illustrative test points only)	0.05–0.20 m	Arm base
IK solver sample range — Y (illustrative test points only)	−0.10–0.10 m	Arm base
IK solver sample range — Z (illustrative test points only)	0.02–0.07 m	Arm base

7. Inverse Kinematic Narrative

An inverse kinematics (IK) pipeline was created with ROS 2 Humble and MoveIt 2 to convert target object positions from the vision subsystem to joint commands. The IK equations were not analytically derived for the 4-DOF chain, but instead MoveIt's built-in kinematic solver was used to compute joint angles for a given end-effector position, given a kinematic model of the robot described in URDF format. The arm has only 4 degrees of freedom, meaning the end effector can't solve for an arbitrary position and orientation (6 degrees of freedom total), so the solver was set to solve for position only, which is appropriate for a pick and place task, and only solved for the position (X, Y, Z) of the end effector without constraining orientation.

7.1 Robot Modelling and MoveIt Setup

The arm was designed in SolidWorks software and exported as a series of STL meshes (base, shoulder, elbow, wrist, end effector) that were combined to form a URDF representing the kinematic chain, joint axes, joint limits, and inertial properties. The URDF has been validated in `robot_state_publisher` and using RViz, which shows the correct loading of the meshes, link orientation and joint hierarchy. The Setup Assistant was then used to create a MoveIt configuration package where the planning group named "arm" is defined as a chain of the base link and the end-effector link. A demo launch created using MoveItConfigsBuilder launched

RViz with the Motion Planning panel and manually commanding the shoulder, elbow and wrist joints verified correct FK and joint-limit behviour.

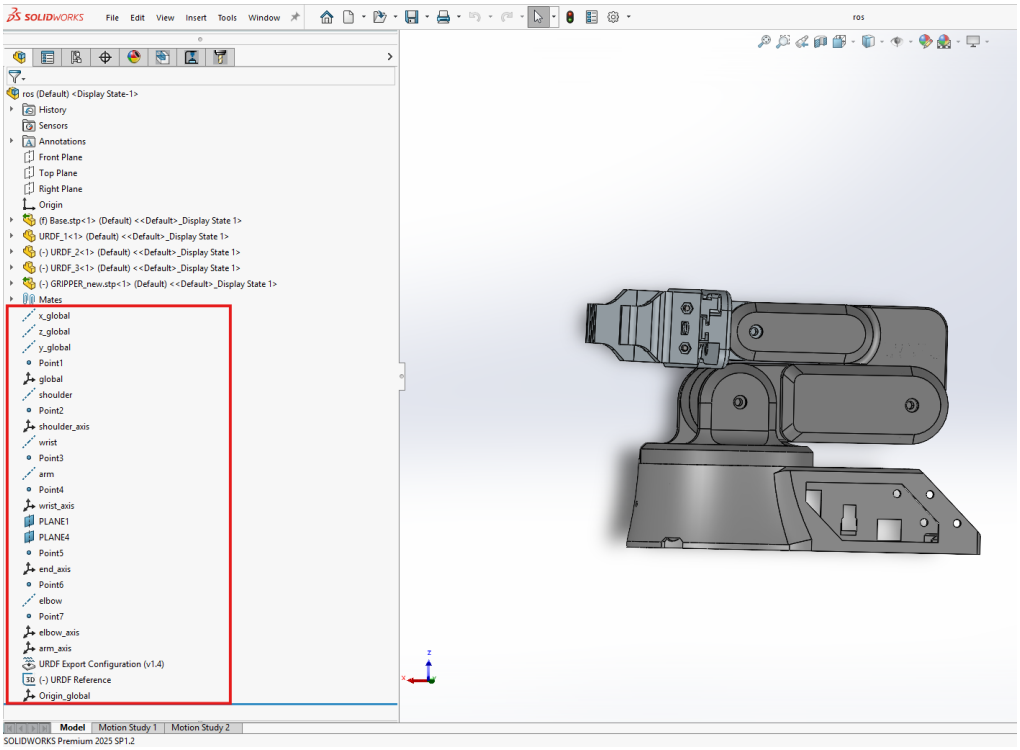

Figure 7-1: SolidWorks used to define joint axis and create the URDF model based on the CAD assembly.

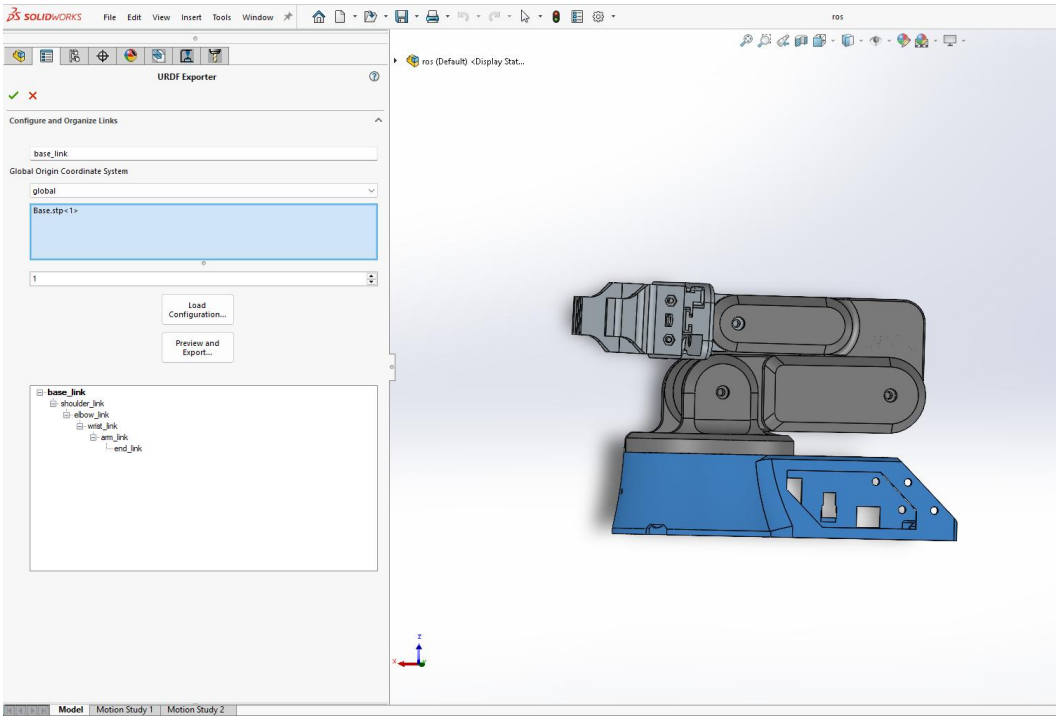

Figure 7-2: Defining the kinematic chain.

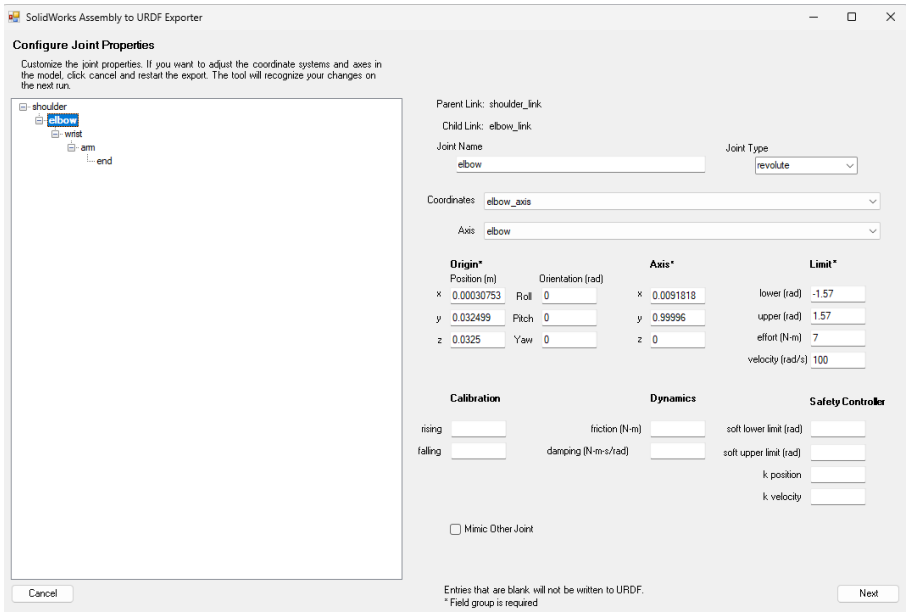

Figure 7-3: Defining joint limits for the URDF model.

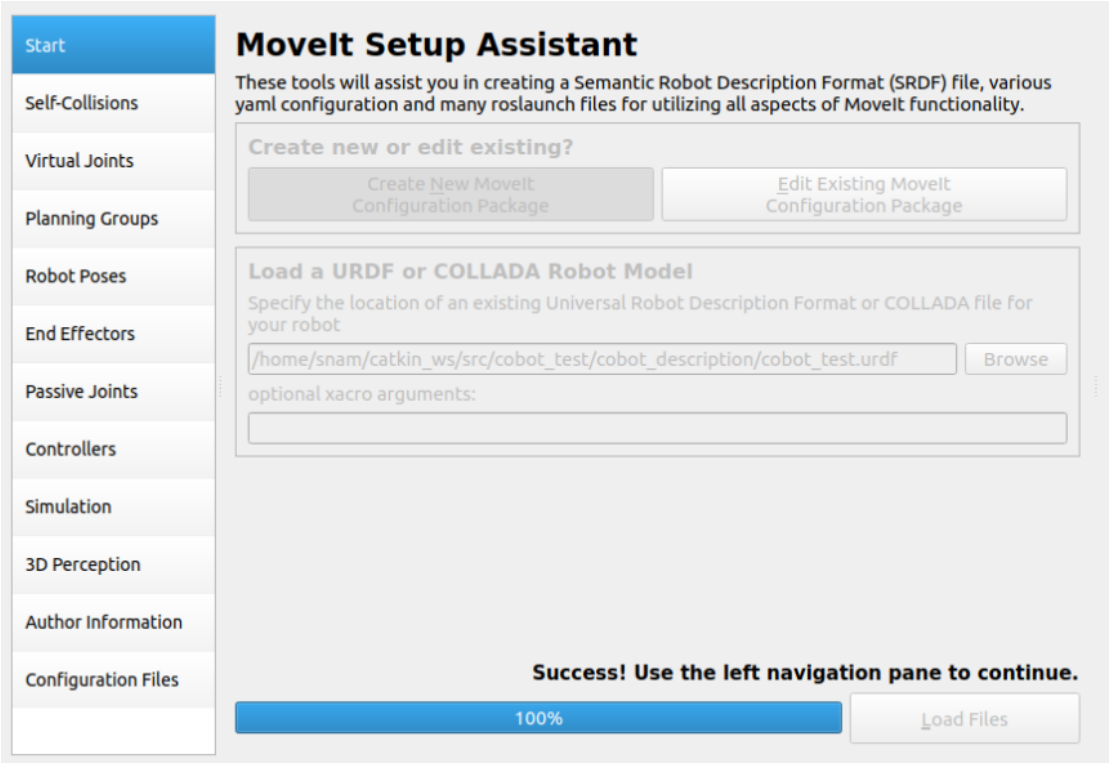

Figure 7-4: MoveIt2 Setup Assistant.

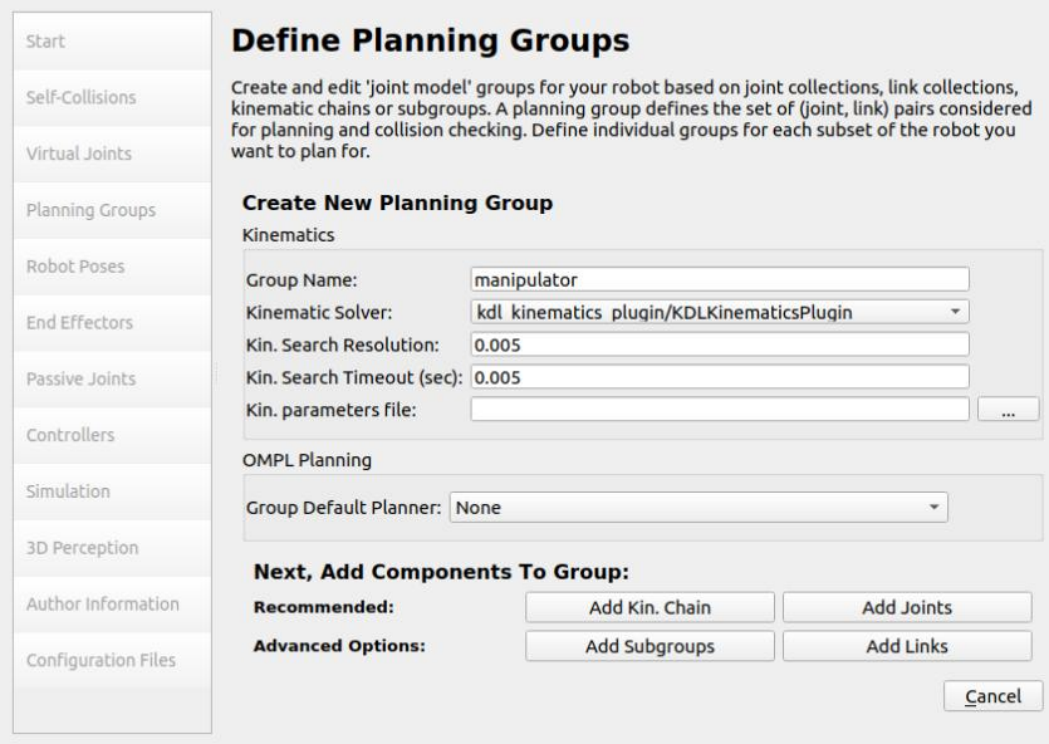

Figure 7-5: Configuring the MoveIt2 solver.

To verify that the joints are working as intended joint_state_publisher was used to independently control individual joints by moving a slider to an angle within the joint limit. Essentially this is forward kinematics, where the joints angles are known and the resulting output needs to obtained.

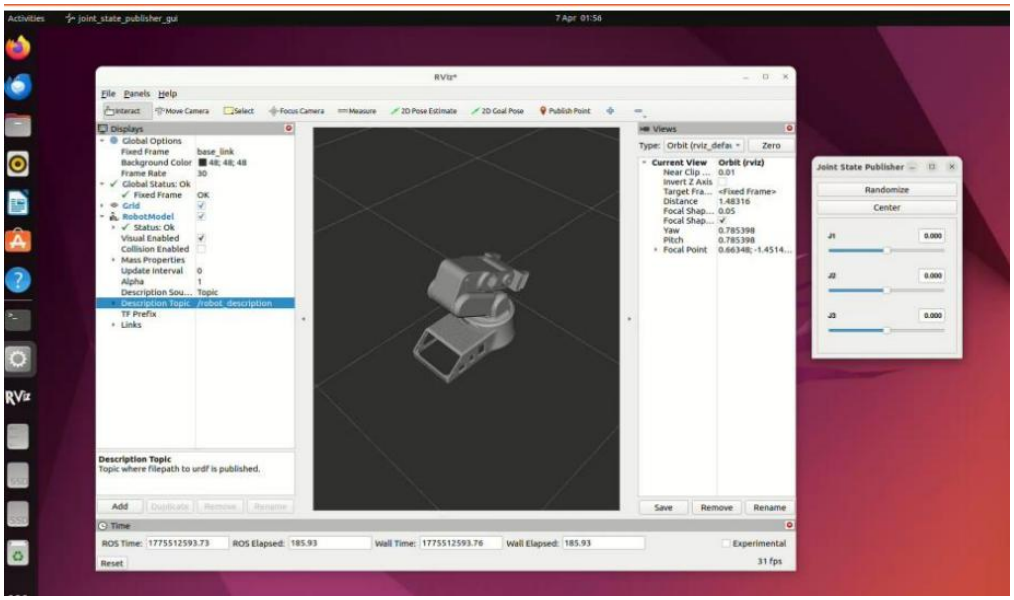

Figure 7-6: Independent control of individual joints using sliders to determine angular position.

7.2 Resolving Initial IK Failures

However, the first attempt at motion planning failed with the error message: “No kinematics plugins defined. Fill and load kinematics.yaml”, despite having specified the KDL plugin in kinematics.yaml. Two causes were found. The first problem was that there was a leftover `tool0` link from an earlier export of a mesh that created a second root link, beside `base_link`, from which MoveIt was unable to form a single kinematic chain; this was removed and the problem was resolved. Second, the custom node that called the IK solver directly reported “No kinematics solver instantiated for group 'arm'”, which is not the case in the MoveIt demo, where the parameters for the robot_description_kinematics are passed automatically. These were addressed by providing these parameters explicitly in a parameter file and setting the node to use `automatically_declare_parameters_from_overrides`. Another problem in which the robot model wasn't displayed in RViz was identified as a problem with the `robot_description` topic not being published by a modified launch file, and was resolved by reverting to the standard MoveIt launch structure.

7.3 Position-Only IK and Workspace Validation

In the test, RViz interactive marker for 6-DOF pose control moved properly along the Z-axis, although the joints moved properly in their full range. This was not a configuration problem, but was deemed to be a side-effect of the arm's 4 degrees of freedom and the need for position-only IK (`position_only_ik: true`) was confirmed and this was used in the rest of the testing. A migration to TRAC-IK was attempted as an alternative solver, but was not possible because the `TRAC_IKKinematicsPlugin` was not found in the installed ROS 2 distribution. A node to test the solver, called workspace_scan.cpp, was created to sweep a grid of Cartesian (X, Y, Z) points and ask the IK solver if they were reachable or not. The parameter loading fix above resulted in a scan reporting all 40 sampled points, covering approximately X = 0.05–0.20 m, Y = −0.10–0.10 m, and Z = 0.02–0.07 m This reached 100% shows that the solver was generating valid joint solutions over the area of interest for pick and place operations, and sets the practical bounds of where objects can be detected and still be picked up.

Table 7-1: Sample of confirmed reachable workspace points.

X (m)	Y (m)	Z (m)	Result
0.05	0.00	0.02	Reachable
0.10	0.05	0.07	Reachable
0.20	0.10	0.07	Reachable
0.15	−0.05	0.04	Reachable
0.08	−0.10	0.05	Reachable

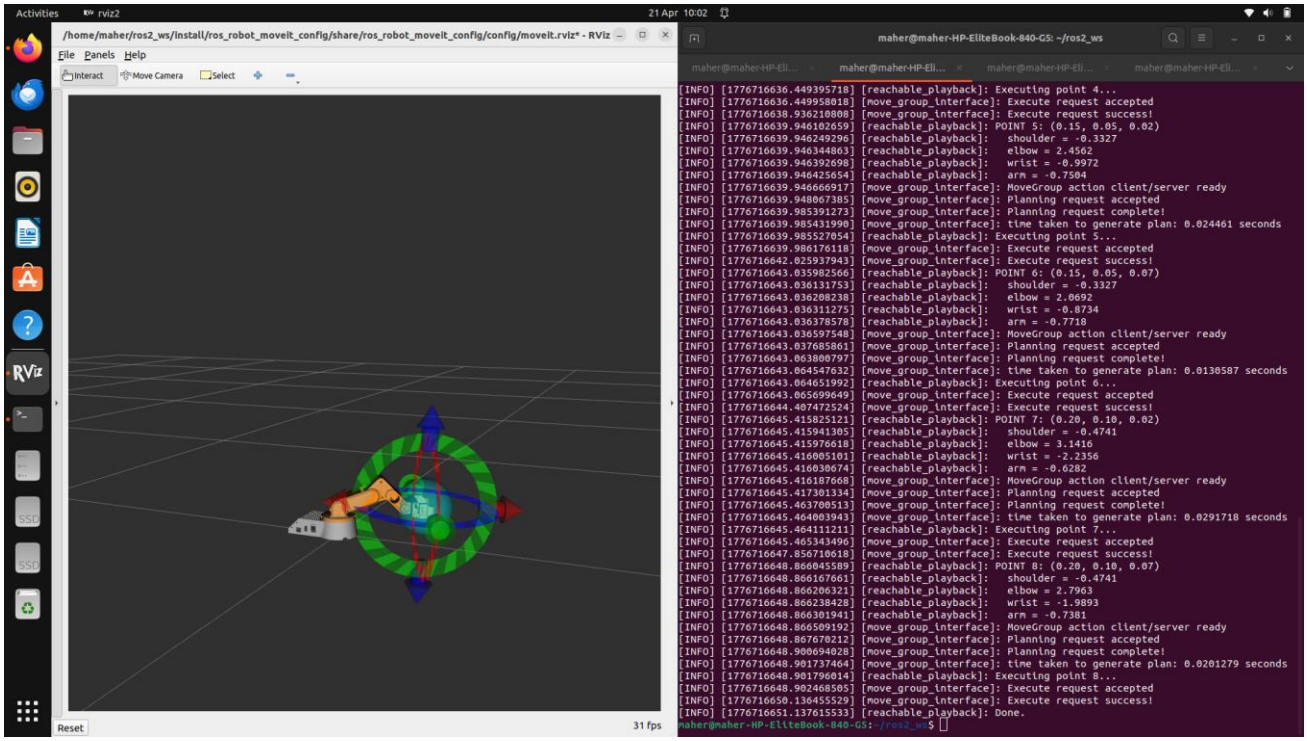

Figure 7-7: Workspace scan to validate inverse kinematic solver.

A final script, called `reachable_playback.cpp`, sends the arm to known reachable Cartesian positions, displays the resulting motion in RViz, and prints the resulting joint angles, closing the control loop from a target Cartesian coordinate, through position-only IK, to a set of joint angles.

The vision subsystem returns the object coordinates from the YOLOv8 and HSV-based detection to the Cartesian-to-joint-angle pipeline, which outputs joint angles that are then converted to PWM tick values and sent to the PCA9685 through the Arduino and the arm is controlled by the PWM tick output. The final step to link the simulated pipeline to the hardware

is to map the MoveIt's radian joint output to the already calibrated ranges of the physical arm, such as the elbow's 150–600 range and the wrist's extended 570–1200 range.

8. Version 2.0 and Lesson Learned

Throughout the development of the Pick and Place Robot, there were several technical challenges that were faced. These provide insight into the design, testing, integration and implementation process therefore reflecting on these experiences can aid in identifying areas of improvement and key considerations that should be made if this project were to be repeated in the future. This section demonstrates the importance of planning, iterative testing and effective component selection to achieve a successful final system.

1. Maintain Backup Options for Critical Components

Alternative components should be evaluated before the final procurement in case project requirements are not met with the current component, this reduces delays within the project. This can be seen with the initial camera selected for the system that needed to be changed after testing was conducted, identifying compatible backup options in advance would have reduced development time and minimised disruptions.

2. Ensure Mechanical Alignment and Precision

Making sure that there are less misalignments within the joints and other structures can significantly affect the placement accuracy. Tolerance control alongside careful assembly is vital for consistent performance and being sure about the dimensions through testing and optimisation can ensure that printing does not need to occur very frequently.

3. Allocate Sufficient Time for System Integration

Integrating the hardware with the software may take slightly longer than anticipated as there can be compatibility issues therefore there should be more time set to the side at the end to integrate and troubleshoot the whole system to ensure the project’s success.

4. Cable Management Throughout Testing and Designing

Stress can be placed onto the wires and cables within the system which can lead to faults or damage therefore strain relief mechanisms can be implemented to improve reliability. This also ensures that there is less of a requirement to consistently change wires that do not function as they should.

5. Testing Conducted in Realistic Conditions

Testing predominantly individual components may not show the potential errors that can occur when the whole system is integrated together. Especially when testing the HSV with colour detection, it is important to obtain values from the area in which the testing will take place. Repeated pick and place cycles can identify performance issues within the system.

6. Prototype Critical Features Early

Key functions such as the gripper should be validated with how it fits into the system earlier on if it is more likely to cause issues when it comes to testing. Early prototyping helps in identifying feasibility issues before other resources are invested into.

7. Maintain Detailed Documentation Throughout Development

Recording vital aspects of the development process such as design decisions, wiring diagrams and code used makes troubleshooting a lot simpler down the line therefore modifications that are required will not be as hard to conduct with a history of all decisions made documented. This also ensures that there is project continuity and easier knowledge transfer to other members of the group.

8. Invest in Higher-Quality Materials and Components

Using higher quality components can improve durability, precision and reliability within the system. The way in which this can happen is by investing in more expensive components to reduce wear, poor accuracy or premature failure. This can ensure that long-term maintenance requirements are reduced therefore improving performance of the system. Investing in more expensive materials can improve the overall quality of the robot alongside prevent alterations later on.

9. References

[1] Arduino, "Arduino Uno Rev3," Arduino Documentation, 2026. [Online]. Available: https://docs.arduino.cc/hardware/uno-rev3. [Accessed 15 June 2026].

[2] STMicroelectronics, "NUCLEO-F446RE," STMicroelectronics, 2026. [Online]. Available: https://www.st.com/en/evaluation-tools/nucleo-f446re.html. [Accessed 15 June 2026].

[3] E. Systems, "ESP32 Series Datasheet," Espressif Documentation, 2026. [Online]. Available: https://www.espressif.com/en/products/socs/esp32. [Accessed 15 June 2026].

[4] Logitech, "C270 HD Webcam Specifications," Logitech Support, 2026. [Online]. Available: https://www.logitech.com/en-us/products/webcams/c270-hd-webcam.html. [Accessed 15 June 2026].

[5] R. P. Foundation, "Camera Module 2," Raspberry Pi Documentation, [Online]. Available: https://www.raspberrypi.com/products/camera-module-v2/. [Accessed 15 June 2026].

[6] InnoMaker, "USB Camera Module Product Listing," Amazon.com, 2026. [Online]. Available: https://www.amazon.com/. [Accessed 15 June 2026].

[7] R. P. Foundation, "Raspberry Pi 4 Model B Specifications," Raspberry Pi Documentation, 2026 2026. [Online]. Available: https://www.raspberrypi.com/products/raspberry-pi-4-model-b/. [Accessed 15 June 2026].

[8] NVIDIA, "Jetson Nano Developer Kit," NVIDIA Developer, 2026. [Online]. Available: https://developer.nvidia.com/embedded/jetson-nano-developer-kit. [Accessed 15 June 2026].

[9] FriendlyElec, "NanoPi M4 Specifications," FriendlyElec Wiki, 2026. [Online]. Available: http://wiki.friendlyarm.com/wiki/index.php/NanoPi_M4. [Accessed 15 June 2026].

[10] TowerPro, "MG995 Servo Motor Datasheet," TowerPro Product Documentation, 2026. [Online]. Available: https://www.towerpro.com.tw/. [Accessed 15 June 2026].

[11] RobotShop, "MG995 Servo Motor, RC Servo, Metal Gear Servo," MG995 Servo Motor, RC Servo, Metal Gear Servo, 2026. [Online]. Available: https://www.robotshop.com/. [Accessed 15 June 2026].

10. Appendix

For CAD, codes, files, and other material please refer to the group GitHub Repository:

https://maherrrrr99.github.io/Group1/